Journal of Network and Computer Applications 236 (2025) 104116

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Review article

A review of graph-powered data quality applications for IoT monitoring sensor networks

Pau Ferrer-Cid *, Jose M. Barcelo-Ordinas, Jorge Garcia-Vidal

Computer Architecture Department, Universitat Politècnica de Catalunya, Barcelona, Spain

ARTICLE INFO

Keywords:
Data quality
Graph signal processing
Graph neural networks
Internet of Things
Machine learning
Monitoring sensor networks

ABSTRACT

The development of Internet of Things (IoT) technologies has led to the widespread adoption of monitoring networks for a wide variety of applications, such as smart cities, environmental monitoring, and precision agriculture. A major research focus in recent years has been the development of graph-based techniques to improve the quality of data from sensor networks, a key aspect of the use of sensed data in decision-making processes, digital twins, and other applications. Emphasis has been placed on the development of machine learning (ML) and signal processing techniques over graphs, taking advantage of the benefits provided by the use of structured data through a graph topology. Many technologies such as graph signal processing (GSP) or the successful graph neural networks (GNNs) have been used for data quality enhancement tasks. This survey focuses on graph-based models for data quality control in monitoring sensor networks. In addition, it introduces the technical details that are commonly used to provide powerful graph-based solutions for data quality tasks in sensor networks, such as missing value imputation, outlier detection, or virtual sensing. To conclude, different challenges and emerging trends have been identified, e.g., graph-based models for digital twins or model transferability and generalization.

Contents

* Corresponding author.
E-mail address: pau.ferrer.cid@upc.edu (P. Ferrer-Cid).

https://doi.org/10.1016/j.jnca.2025.104116
Received 28 October 2024; Received in revised form 21 December 2024; Accepted 19 January 2025
Available online 28 January 2025

1. Introduction

The GSP field emerged to translate the classic signal processing concepts, related to the analysis of time-dependent signals, to irregular domains such as graphs, defining the notion of signals defined over graphs (Shuman et al., 2013; Sandryhaila and Moura, 2013a). The use of graph-based models had already been a key element in applications such as route planning (e.g., Dijkstra's algorithm) (Delling et al., 2009), community detection (e.g., clique percolation or Louvain algorithms) (Fortunato, 2010) or the analysis of complex networks such as biological and social networks (Pavlopoulos et al., 2011; Newman et al., 2002). The representation of manifolds by means of graphs in the field of semi-supervised learning is another example of graph-powered applications (Belkin and Niyogi, 2004). Overall, the GSP framework (Leus et al., 2023) has enabled the use and development of novel techniques on data residing in graphs, thus emerging as an alternative to classical ML techniques that do not make explicit use of data structure. In this way, the graph topology, which represents the relationships between the graph's nodes, is fed to graph-based models that explicitly model the structure of the data (Dong et al., 2020).

A wide variety of concepts have been applied to signals defined over graphs, such as signal shift, translation, convolution, or filtering (Ortega et al., 2018). An important concept of the GSP is the notion of signal smoothness, also expressed via the total variation (TV) or the Dirichlet energy, which are quadratic forms and can be used to evaluate how a signal fits a given graph structure or vice-versa (Zhang et al., 2015). This idea is linked with the Graph Discrete Fourier Transform (GDFT) that makes use of the graph topology to obtain the graph Fourier basis and allow the computation of the transform coefficients of a graph signal and also led to the development of graph filters (Sandryhaila and Moura, 2013b). In the field of ML, graphs have been used as regularizers in optimization problems, e.g., the regularization of neural networks for semi-supervised learning tasks (Chami et al., 2022). Moreover, the structured representation of the data allows for improving the interpretability of the models as well as the interpretability of their results (Dong et al., 2020). Given the growing popularity of black-box models (e.g., deep neural networks), much emphasis has been placed on the interpretability of the techniques to facilitate their use. In addition, one of the greatest successes of graph-based techniques has been the development of GNNs, allowing the use of neural networks on data residing in irregular domains such as graphs, from more general message-passing-based GNNs to spectral networks based on the graph convolution operation (Zhou et al., 2020; Wu et al., 2022).

Recently, graphs have unveiled their potential to represent IoT monitoring sensor networks, allowing the use of signal processing and ML techniques to carry out applications using sensor network data (Jabłoński, 2017; Dong et al., 2023). Monitoring sensor networks are platforms whose purpose is to monitor the evolution of certain phenomena in a specific area of interest or scenario, e.g., precision agriculture systems (López et al., 2022), air quality monitoring (Ferrer-Cid et al., 2020), or industrial processes (Wu et al., 2021a; Su et al., 2024). These networks are of particular interest because they monitor critical phenomena or assets and provide data for further applications or decision-making processes. Fig. 1 depicts different scenarios where IoT monitoring networks are used, common use cases include smart metering, industrial applications, and environmental monitoring among others.

Data-driven IoT applications aim to exploit the network measurements as well as their intrinsic structure to carry out several tasks. The quality of data from monitoring sensor networks is critical since the data can feed various applications or decision-making processes that require complete and accurate data. In the literature, data quality tasks have been developed to provide continuous, complete, and accurate data. Data quality in IoT platforms is defined by different dimensions such as data accuracy, sample completeness, data consistency or coherence, and data timeliness (Guerra-García et al., 2023; Mansouri et al., 2023; Liu et al., 2020a). Some of the issues that may affect such data attributes are the presence of missing data, outliers, inconsistent samples, etc. Therefore, some of the tasks developed include the detection of outliers and anomalies (Ferrer-Cid et al., 2022b), the use of predictive models to create virtual sensors (Yan et al., 2022), the imputation of missing data (Jiang et al., 2021), and the detection of clusters and communities among others (Jabłoński, 2017). In the context of sensor data, models have also been developed to estimate the reliability of the measurements, an aspect related to possible measurement deviations and thus outlying or anomalous measurements (Shafin et al., 2024). Indeed, graph-based models provide alternatives to classic data-driven approaches and have been proven to be superior in terms of performance, possibility of distributed approaches, and interpretability. IoT data is characterized by the presence of spatiotemporal correlations given that monitoring platforms measure phenomena in an area of interest or assets in an industrial environment. In particular, graph-based models are known for their ability to model complex relationships between sensors and leverage spatiotemporal correlations (Dong et al., 2023). Henceforth, a deep understanding of signal processing and ML over graph approaches may unveil new applications and advantages for IoT monitoring sensor networks.

Recent surveys are based on the technical review of models based mainly on GSP and GNN, mentioning some applications in areas such as computer vision or natural language processing (Zhou et al., 2020; Song et al., 2022; Wu et al., 2020; Jin et al., 2023; Xia et al., 2021). Others focus on the applications of GNNs and graph-based models in areas such as the IoT, showing graph-based applications in cases such as environmental monitoring or biological data (Dong et al., 2023; Li et al., 2023b, 2021c; Wang et al., 2023). In this survey, we focus on the application of graph-based models to the recently booming area of data quality improvement for monitoring IoT platforms (Ferrer-Cid et al.,

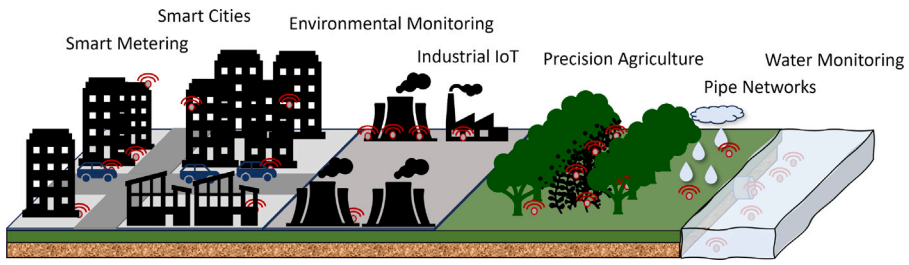

Fig. 1. Examples of different applications where IoT sensor networks are leveraged to monitor different phenomena. Recent use cases include applications in air quality networks, precision agriculture, or pipe networks among others.

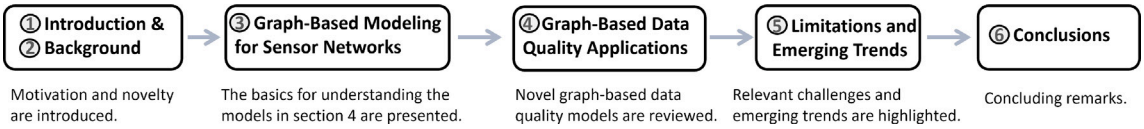

Fig. 2. Outline of the survey. Section 3 introduces the basics of the different graph-based approaches (GSP, ML over graphs, and GNNs) on which the methods discussed in Section 4 are based.

2024a; Wang et al., 2023). In particular, we not only cover GNNs applications but also provide complete coverage of graph-based modeling of monitoring networks, covering GSP, ML over graphs, and GNNs. We also present recent advances and applications in data quality control and provide a taxonomy for the most novel graph-based applications and trends.

The outline of this paper is as follows; Section 2 motivates the content of the paper. Section 3 introduces technical concepts regarding signal processing and ML over graphs. Section 4 reviews different graph applications in IoT monitoring sensor networks, and Section 5 presents challenges, limitations, and emerging trends. Finally, Section 6 concludes the manuscript. Fig. 2 shows the outline of the survey.

2. Background and motivation

In this section, we revisit the development of graph-based techniques and we motivate the use of graph-based models for IoT data quality applications.

Before the appearance of the GSP, graphs were already used in the field of graph-based semi-supervised learning to present the relationships of data residing in a manifold (Song et al., 2022; Subramanya and Talukdar, 2022). In fact, label propagation emerged as a transductive method to estimate unlabeled data points from labeled ones, and inspired the GNN framework (Wang and Leskovec, 2020). The advent of the GSP was a revolution since it allowed the translation of solid signal processing fundamentals to the realm of signals defined over graphs. Thus, this allowed the analysis of measurements residing on a graph via graph filters, signal shifts, signal convolutions, or the spectral graph Fourier transform among others. In turn, the graph signal convolution led to the translation of convolutional neural networks to the graph paradigm, making use of the graph signal convolution operator, e.g., the Chebnet GNN (Defferrard et al., 2016). Besides all the guarantees in terms of efficiency and accuracy, the GSP-based techniques provide interpretability, which has received a lot of attention due to the growing use of black-box models, given the use of the graph shift operator, i.e., the matrix that describes the relationships between network nodes. Moreover, many of the techniques, e.g., graph filters, allow a distributed implementation, which is very interesting in case of resource limitations or in case of having nodes with computational resources, e.g., edge computing approach (Coutino et al., 2019). All this together makes the GSP provide appropriate tools to analyze the measurements produced by a network of sensors, being able to define these measurements as signals that reside in a graph.

ML over graphs usually refers to ML techniques that have been adapted to take into account that a graph can represent the data, or analogously that the data resides in a manifold. In this sense, graph-based semi-supervised learning is aimed at the application of algorithms to partially-labeled data residing on manifolds represented by graphs (Belkin and Niyogi, 2004). This idea has also been used in different works where the optimization function of different ML techniques has been modified to include a regularization term that takes into account that the data points reside on a graph (Hang et al., 2016). Among some examples, we can find predictive models, e.g., kernel ridge regression (Venkitaraman et al., 2019) or regularized support vector regression (Li et al., 2020a), which can be very useful in the case of sensor network measurements. Even though, there exists a large range of tasks and applications that can be approached using ML, e.g., the target of the ML task can be estimating features at graph nodes, at graph edges, or at the graph as a whole. Throughout the manuscript, we restrict to the tasks that are useful in the sensor network domain.

Although GNNs belong to the field of ML over graphs, in this paper we want to dedicate their own space to them due to their great importance and widespread adoption nowadays. Some well-known architectures include the GraphSAGE (Hamilton et al., 2017) or the Chebnet (Defferrard et al., 2016). Indeed, the graph convolution operator allowed the application of the convolution neural networks to the graph realm, resulting in graph convolutional neural networks such as the Chebnet, in which the convolution is realized via a graph filter. Given the widespread application of neural networks, tailored GNN architectures are being continuously developed to tackle tasks such as missing value imputation, anomaly detection, data compression, event detection, or signal reconstruction (Zhou et al., 2020). Another example of disruption in this field is the emergence of graph attention networks and graph transformers, being the analogs of attention mechanisms and transformers in conventional neural networks (Ahmad et al., 2021).

Recently, graphs have been used to model measurements from IoT platforms in industrial, smart city, or precision agriculture applications among others (Dong et al., 2023). In this way, structured data modeling using graphs has been exploited, either by GSP or GNN (Ortega et al., 2018; Egilmez and Ortega, 2014; Wu et al., 2023). Some of the applications include structure learning, multivariate time series forecasting, and event or anomaly detection (Calo et al., 2024; Chen et al., 2021a; Xiao et al., 2020a). Recent studies have investigated the use of GNN in IoT applications (Dong et al., 2023; Li et al., 2023b; Wang et al., 2023). Nevertheless, although there are many methods intended for data quality tasks, there is no survey that covers all these methods and frameworks in the context of data quality improvement in IoT monitoring networks.

Fig. 3 depicts the scope of this review. From now on we focus on GSP, ML over graphs, and GNNs, placing special emphasis on novel applications for data quality in monitoring sensor networks. Data quality

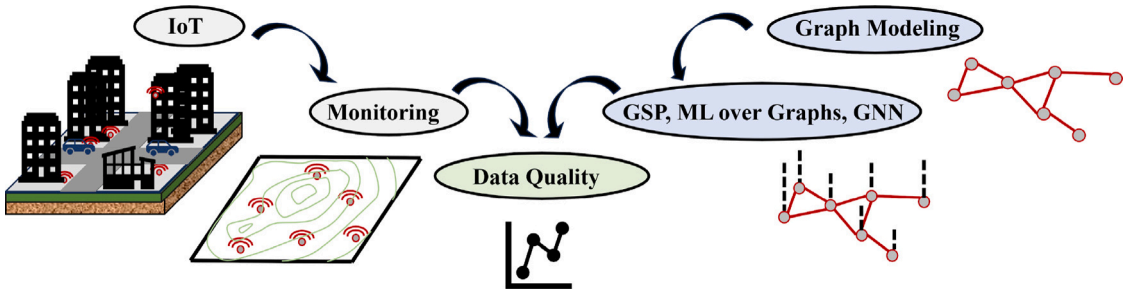

Fig. 3. Scope of the review, focus on graph-based models (GSP, ML over graphs, and GNNs) for data quality tasks in IoT monitoring sensor networks.

Table 1
A comparison between the scope of this survey and the scope of other relevant surveys.

	GSP	ML over graphs	GNN	IoT monitoring	Data quality
Teh et al. (2020) and Mansouri et al. (2023)				✓	✓
Leus et al. (2023), Ortega et al. (2018), Dong et al. (2019) and Xia et al. (2021)	✓				
Chami et al. (2022) and Song et al. (2022)		✓			
Wu et al. (2020, 2022), Zhou et al. (2020), Zhang et al. (2019) and Jin et al. (2023)			✓		
Dong et al. (2020) and Li et al. (2021c)	✓	✓	✓		
Dong et al. (2023), Li et al. (2023b) and Wang et al. (2023)			✓	✓	
Our work	✓	✓	✓	✓	✓

is a key challenge in monitoring sensor networks, with tasks ranging from anomaly detection to missing value imputation (Ferrer-Cid et al., 2024a). Although existing surveys cover various topics related to GSP and GNN for IoT (Table 1), the review of graph-based models for data quality in IoT has not yet been covered. In addition, the survey groups into useful applications (in the context of IoT data quality) different works that have developed methods for different applications in isolation. Hence, this survey provides a novel description of graph-based models for data quality tasks in IoT monitoring sensor networks. In summary, our contributions are:

- describe the foundations of different graph-based models for IoT sensor networks (GSP, ML over graphs, and GNNs) that can be used in data quality enhancement applications,
- review recent work and provide a taxonomy of applications that can help practitioners implement graph-based models in monitoring sensor networks to improve data quality and ensure data quality requirements,
- highlight the challenges and limitations that practitioners may face, as well as identify emerging trends.

3. Graph-based modeling for sensor networks

In this section, we elaborate on the fundamentals of the GSP, ML over graphs, and GNNs, highlighting current trends and novel techniques. We denote the measurement of a given sensor i at a given time t as $x_i(t) \in \mathbb{R}$, N denotes the number of sensors, and $\mathbf{x}_i \in \mathbb{R}^T$ represents a vector of T measurements recorded by the ith sensor. Matrices are denoted by bold uppercase letters, vectors by bold lowercase letters, and sets by calligraphic letters.

Sensor network graph representation: a monitoring sensor network and the relationship between nodes' measurements can be represented as a graph $\mathcal{G}$. In turn, a graph can be fully described by the triplet $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{S}\}$, where $\mathcal{V} = \{1, \dots, N\}$ is the set of nodes representing the sensors, $\mathcal{E} \subseteq \{(i,j)\colon i,j \in \mathcal{V}\}$ denotes the set of edges representing connected nodes, i.e., related sensors, and $\mathbf{S} \in \mathbb{R}^{N \times N}$ is a graph matrix describing the edges $\mathcal{E}$ of the graph, i.e., $\mathbf{S}_{ij} \neq 0$ iff $(i,j) \in \mathcal{E}$. Fig. 4 represents a sensor network described by graph $\mathcal{G}$. Graph structure identification, i.e., constructing the graph representing a sensor network is a research field on its own and it is further explained in Section 3.1.1. While a graph can denote *spatial*, *temporal*, and *spatiotemporal* dependencies between nodes' data, spatiotemporal graphs can be constructed

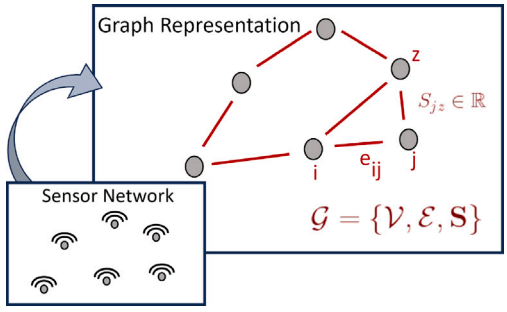

Fig. 4. Example of how a graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathbf{S}\}$ can be used to represent a sensor network and the sensors' relationships. e_{ij} represents an edge connecting nodes i and j. The entry of the graph shift matrix S_{ij} assigns a weight to the edge e_{ij}.

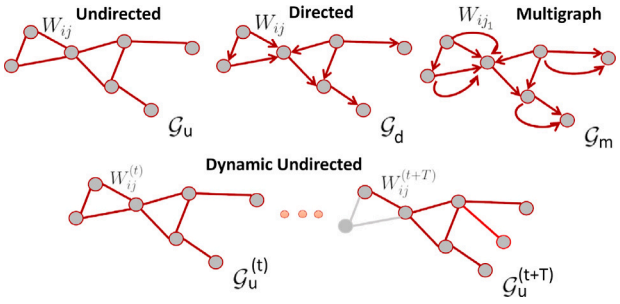

Fig. 5. Types of graphs: above, static graphics; below, dynamic graphics; u denotes undirected, d denotes directed, m denotes multigraph, and $\mathcal{G}_u^{(t)}$ denotes a dynamic undirected graph at time t. $T \in \mathbb{N}$ represents a time offset.

by combining a spatial graph $\mathcal{G}_s$ and a temporal graph $\mathcal{G}_t$ via their cartesian product $\mathcal{G}_{st} = \mathcal{G}_s \times \mathcal{G}_t$.[1] Another possible data-driven approach

[1] There exist several ways to combine the structure of two graphs, e.g., cartesian product or Kronecker product. Refer to Stanković et al. (2020a) for further information on graph combination.

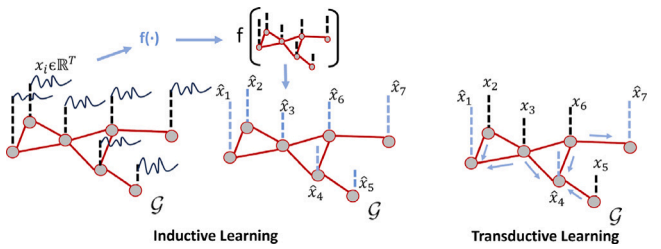

Fig. 6. Inductive models are trained to perform inferences in other test sets while transductive models are trained to perform inferences for a specific test set. In black training data, and in blue testing data.

for learning spatiotemporal graphs consists of rearranging the sensors' measurements so that time lags are included as features.

In addition, there are two variants of graphs that can be applied in sensor networks: *(i) static* graphs $\mathcal{G}$ and *(ii) dynamic* or time-varying graphs $\mathcal{G}^{(t)}$ whose structure depends on time t (Skarding et al., 2021). Static graphs can be used to represent sensor networks that are static or fixed so the relationships between the sensors are not expected to change. In contrast, dynamics graphs have a time-dependent structure, meaning that the relationships between the sensors as well as the set of sensors composing the graph can change over time. This approach is useful to represent mobile sensor networks or dynamic networks where sensors can join and leave the network and the relationship between them can change. We can further classify the graphs into three types; *(i) undirected*, in which the relationships between nodes are reciprocal, i.e., $(i,j) \in \mathcal{E}$ implies $(j,i) \in \mathcal{E}$, or $S_{ij} = S_{ji}$, *(ii) directed*, in which the relationships do not need to be reciprocal, and *(iii) multigraphs*, in which more than one edge can exist for a given pair of nodes (i,j). Fig. 5 represents the different types of graphs. In the specific case of monitoring sensor networks, where a graph structure is used to represent the relationship between the sensors' data, it makes sense to assume that the relationships between the sensor measurements are reciprocal, e.g., as in the case of correlation coefficients. This survey therefore focuses on the use of undirected static graphs, as monitoring platforms are typically fixed after deployment. More complex scenarios, such as mobile monitoring sensor networks, require more complex techniques and the use of dynamic graphs.[2]

In addition, before moving on to explain the technical aspects of signal processing and ML over graphs, it is important to highlight the use of two types of predictive models in this graph-based approach; *(i) inductive* models, and *(ii) transductive* models. The objective of inductive learning is to infer a model from training instances, then this model can be applied to different testing instances. In contrast, in the case of transductive learning, training instances are used to obtain estimates for specific test instances. As we will see later, even though in the realm of sensor networks many models can be used in both ways, there are specific models of both types, such as a graph filter (inductive) or a graph kernel ridge regression (transductive). Fig. 6 illustrates the idea behind these two learning paradigms.

3.1. Graph signal processing

The GSP emerged to adapt the signal processing foundations to signals with irregular support such as graphs (Sandryhaila and Moura, 2013a; Shuman et al., 2013; Ortega et al., 2018). A graph signal is defined as the map $x\colon \mathcal{V} \to \mathbb{R}$, then the ith sensor measurement at a given time instant can be expressed as $x_i(t) \in \mathbb{R}$ and the collection of measurements of the network at a given instant as $\mathbf{x}(t) \in \mathbb{R}^N$ and a collection of measurements for a period of time as $\mathbf{X} = [\mathbf{x}(t_1), \dots, \mathbf{x}(t_T)] \in \mathbb{R}^{N\times T}$. One of the key components of this framework is the graph shift matrix $\mathbf{S}$ which describes the graph topology, and allows us to define the notion of graph shift, the GDFT, as well as the notion of signal smoothness among others. From now on, we simplify the notation of a graph signal at a given instant as $\mathbf{x}(t) \equiv \mathbf{x}$. Fig. 7 illustrates the use of the GSP framework to model the measurements of a sensor network, where the measurements are defined as signals on the graph and the graph shift matrix $\mathbf{S}$ describes the relationships between the sensors.

3.1.1. Shift matrix

The entries of the graph shift matrix S_{ij} are used to describe the relationship between the different pairs of graph nodes (i,j). The graph shift matrix is used as signal shift operator (Gavili and Zhang, 2017):

$$\mathbf{x}^{(1)} = \mathbf{S}\mathbf{x} \tag{1}$$

Hence, the graph shift is defined as the linear combination of the components of a graph signal $\mathbf{x} \in \mathbb{R}^N$ weighted by the entries of the shift matrix, Fig. 8. There exist different representations for the graph shift matrix $\mathbf{S}$, among the most common we find; *(i)* graph *adjacency* matrix $\mathbf{A} \in \mathbb{B}^{N\times N}$ which indicates the existing edges in the graph ($\mathbf{S} = \mathbf{A}$), such that $A_{ij} = 1$ iff $(i,j) \in \mathcal{E}$ and $A_{ij} = 0$ otherwise, *(ii)* the graph *weight* matrix $\mathbf{W} \in \mathbb{R}^{+^{N\times N}}$ which assigns a non-negative weight to the graph edges ($\mathbf{S} = \mathbf{W}$), i.e., $W_{ij} > 0$ iff $(i,j) \in \mathcal{E}$, and *(iii)* the combinatorial *Laplacian* matrix $\mathbf{L} \in \mathbb{R}^{N\times N}$ ($\mathbf{S} = \mathbf{L}$), that is defined as $\mathbf{L} = \mathbf{D}-\mathbf{W}$, where $\mathbf{D} = diag(\mathbf{W1})$, where $\mathbf{1} \in \mathbb{R}^N$ is a vector of ones. In fact, the graph shift matrix expresses the dependencies and independencies between graph nodes and it is usually forced to be sparse in order to capture the most critical relationships within the graph and to take advantage of the computational benefits of manipulating sparse matrices (Bunch and Rose, 2014). Other graph shift matrices can be found in Shuman et al. (2013).

To be able to define graph filters, we need to adapt the notion of classical signal frequency spectrum to the graph domain. For this purpose, the GDFT is used to provide the frequency spectrum of a graph signal via the graph shift matrix eigendecomposition (Ricaud et al., 2019). The eigendecomposition of matrix $\mathbf{S}$ is defined as:

$$\mathbf{S} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^{-1} \tag{2}$$

where the eigenvector matrix $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N] \in \mathbb{R}^{N\times N}$ defines the components of the Fourier modes and $\boldsymbol{\Lambda} = diag([\lambda_1, \dots, \lambda_N]) \in \mathbb{R}^{N\times N}$ includes the frequencies defined by the eigenvalues. For undirected graphs, the matrix $\mathbf{S}$ is real symmetric so $\mathbf{U}^{-1} = \mathbf{U}^{\mathrm{T}}$. Slow-varying components are associated with small eigenvalues while fast-changing components are associated with large eigenvalues. In this way, the GDFT can be defined as (Sandryhaila and Moura, 2013b):

$$\hat{\mathbf{x}} = \mathbf{U}^{-1}\mathbf{x} \tag{3}$$

where $\hat{\mathbf{x}} \in \mathbb{R}^N$ is the GDFT transform of $\mathbf{x}$. This aspect will be further reviewed in the next Section 3.1.3. As we have seen, the graph shift matrix is a core element in this graph-based approach, because of that, there exists extensive literature tackling the graph learning problem (Mateos et al., 2019; Dong et al., 2019). The goal of the graph learning task is to construct a graph, i.e., to build the matrix $\mathbf{S}$ (which in turn defines the set of edges $\mathcal{E}$), that best describes the relations in the network. Here, we enumerate the two most common approaches:

- *Graph based on prior information*: the graph is constructed using some prior information. An example can be the use of the geographical distance, where a weighting function based on the nodes' geographical distance is used (Sandryhaila and Moura, 2013a). For instance, the Gaussian radial basis function is commonly used to assign weights, $W_{ij} = e^{\frac{-d_{ij}}{2\sigma}}$, where $d_{ij} \in \mathbb{R}$ is the geographical distance between nodes i and j. Then, a threshold $TH \in \mathbb{R}$ can be used to force a certain graph sparsity. Another commonly used approach to restricting the sparsity of the resulting graph is building a K-nearest neighbor approach,

[2] Refer to Yan et al. (2024) for a systematic review of dynamic graphs.

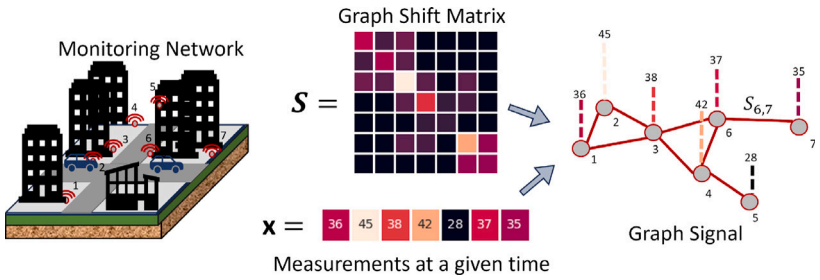

Fig. 7. The GSP framework defines sensor network measurements as signals defined over graphs.

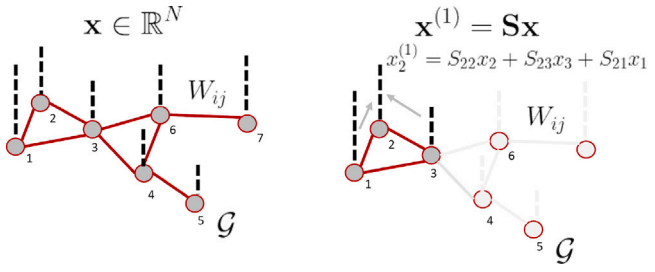

Fig. 8. Representation of a graph signal $\mathbf{x} \in \mathbb{R}^N$ over a graph $\mathcal{G}$ and the effect of the graph shift operation via the graph shift matrix $\mathbf{S}$. The graph shift operator can be seen as a linear combination of the graph signal evaluated at neighboring nodes weighted by the entries of the shift matrix.

i.e., only keeping the K most important edges per node. Some other examples may include the construction of the adjacency matrix according to the connectivity between sensors, i.e., $A_{ij} = 1$ iff sensors i and j have communication.

- *Data-driven graph*: the graph shift $\mathbf{S}$ is learned from a given set of graph signals $\mathbf{X} \in \mathbb{R}^{N \times T}$ so that the resulting graph fits the collected measurements. The general data-driven graph learning problem can be defined as an optimization problem:

$$\min_{\mathbf{S} \in \mathbb{R}^{N \times N}} L(\mathbf{X}, \mathbf{S}) + \lambda \cdot R(\mathbf{S}) \tag{4}$$

$$s.t. \quad \mathbf{S} \in \mathcal{S} \tag{5}$$

where the function $L(\mathbf{X}, \mathbf{S})$ evaluates the goodness-of-fit of the graph with respect to the set of measurements $\mathbf{X}$ and $R(\mathbf{S})$ is a regularization function that controls the complexity of the solution and is governed by a hyperparameter $\lambda \in \mathbb{R}$. The constraint forces the graph shift matrix $\mathbf{S}$ to belong to the set of valid graph shift matrices $\mathcal{S}$, whose properties depend on the specific matrix used, e.g., Laplacian matrix, normalized Laplacian matrix, weight matrix, etc.

Different graph learning problems have been formulated, some of them make use of the signal smoothness criterion as function $L(\cdot)$, and others use different regularization functions $R(\cdot)$ to force a specific graph connectivity. For instance, Dong et al. (2019) review different graph learning methods, from latent factor models to models based on graph filters and graph dictionaries. In fact, Dong et al. (2016) develop a factor analysis-based model to learn the Laplacian matrix $\mathbf{L}$ using the smoothness criterion along with a Frobenius norm. Kalofolias (2016) propose an optimization model to learn the graph shift matrix $\mathbf{W}$. Other examples include the learning of valid precision matrices, or GNNs to learn a graph representation and a GNN model jointly (Egilmez et al., 2017; Xia et al., 2021). For instance, Ferrer-Cid et al. (2021) compare the use of data-driven (graph signal processing-based and statistics-based) and distance-based graph learning methods for air quality monitoring networks. Moreover, algorithms have been developed to learn time-varying graphs, allowing the graph to change over time according to the set of samples (Mateos et al., 2019).

This graph learning task plays an important role in IoT monitoring sensor networks, where the graph is usually fed to an optimization model to carry out a specific task, e.g., missing value imputation. Thus, the correct identification of the graph structure, i.e., the relationships within the data, can impact the performance of the final task.

3.1.2. Signal smoothness

Signal smoothness is a widely adopted concept in optimization problems and applications. Broadly speaking, a graph signal $\mathbf{x} \in \mathbb{R}^N$ is said to be smooth with respect to an underlying graph if the graph strongly connects nodes with similar measurements and dissimilar nodes are weakly connected or disconnected. Formally, the signal smoothness is usually evaluated through the quadratic form (Stanković et al., 2020a):

$$S(\mathbf{S}, \mathbf{x}) = \mathbf{x}^{\mathsf{T}} \mathbf{S} \mathbf{x} \tag{6}$$

This metric has been widely used in problems such as graph learning, outlier detection, and missing value imputation (Gopalakrishnan et al., 2019; Dong et al., 2016). In the case $\mathbf{S} = \mathbf{L}$ then $S(\mathbf{L}, \mathbf{x}) = \frac{1}{2} \sum_{i,j \in \mathcal{V}} W_{ij} (x_i - x_j)^2$ = meaning that the smaller the Laplacian quadratic form the more smooth the signal is. Nevertheless, we must place special care when using this criterion since its minimum is achieved for a total disconnected graph $\mathcal{E} = \varnothing$ or a constant or null graph signal $\mathbf{x} = \mathbf{0}$. Analogously to classic signal processing, the graph TV can be defined through the definition of the graph shift operator $\mathbf{S}$ (Sandryhaila and Moura, 2014b):

$$\mathrm{TV}_p(\mathbf{S}, \mathbf{x}) = \|\mathbf{x} - \mathbf{S}\mathbf{x}\|_p \tag{7}$$

where $p \in \mathbb{N}^+$ and $\|\cdot\|_p$ denotes the l^p vector norm. This metric has also been used for signal reconstruction as well as for graph signal classification since it is a global graph metric, i.e., signal-wise (Chen et al., 2015).

Other metrics exist to evaluate the variation of a graph signal, such as the local variation of a graph signal or the discrete p-Dirichlet energy form. Recently, the Sobolev smoothness for graph signals has been used for applications such as signal reconstruction for missing value imputation in sensor networks. The Sobolev smoothness for an undirected graph can be defined as (Giraldo et al., 2022):

$$S_{\beta,\epsilon}(\mathbf{L}, \mathbf{x}) = \mathbf{x}^{\mathsf{T}} (\mathbf{L} + \epsilon \mathbf{I})^{\beta} \mathbf{x} \tag{8}$$

where $\epsilon \in \mathbb{R}$ and $\beta \in \mathbb{R}$ are hyperparameters. In fact, the minimization of a graph signal smoothness can be translated to the penalization of the GDFT components $\hat{\mathbf{x}}_i$ according to the frequencies λ_i, being the largest frequencies the most penalized ones (Giraldo et al., 2022).

3.1.3. Graph filters

Now that we have explained the use of the graph shift matrix as well as the utility of its eigendecomposition to formulate the GDFT, we can formulate the graph filters. We assume that the eigenvectors $\mathbf{u}_i$ are ordered in ascending order according to the eigenvalues, $\lambda_1 \leq \cdots \leq \lambda_N$. Given the definition of the GDFT (Sandryhaila and Moura, 2013b), a non-parametric filter can be defined as a function of the eigenvalues in frequency domain, or a function of the shift matrix in

vertex domain (Sandryhaila and Moura, 2014b):

$$\tilde{\hat{\mathbf{x}}} = g(\boldsymbol{\Lambda})\hat{\mathbf{x}} \tag{9}$$

$$\tilde{\mathbf{x}} = \mathbf{U}g(\boldsymbol{\Lambda})\mathbf{U}^{-1}\mathbf{x} = f(\mathbf{S})\mathbf{x} \tag{10}$$

More precisely, a linear graph filter can be defined in the vertex domain as (Sandryhaila and Moura, 2013a):

$$\tilde{\mathbf{x}} = \sum_{i=0}^{K-1} h_i \mathbf{S}^i \mathbf{x} \tag{11}$$

where $\mathbf{h} \in \mathbb{R}^K$ are the filter's taps. The filtered version $\tilde{\mathbf{x}}$ of the signal $\mathbf{x}$ is a linear combination of the shifted versions of the signal, $\mathbf{S}^k\mathbf{x}$. Other filters can be used in the vertex domain to carry out applications such as denoising, signal reconstruction, or anomaly detection, as well as frequency-vertex domain representations using graph wavelet transforms or graph slepians (Xiao et al., 2020b; Van De Ville et al., 2017). In fact, there exists a wide variety of filters, from linear to nonlinear, which allow the implementation of powerful predictive models capable of reconstructing the signal at any of the sensors of a network. Examples of filters include convolutional filters (Wu et al., 2019), nonlinear polynomial graph filters (Xiao et al., 2020b), Volterra-like filters (Xiao et al., 2021), etc.

To approximate the computation of the polynomials of $\mathbf{L}$ and $\boldsymbol{\Lambda}$ for a specific transfer function, different bases can be used, e.g., the Chebyshev polynomials, the Gegenbauer polynomials, and the monomial or Bernstein basis among others (He et al., 2021; Puny et al., 2023; Castro-Correa et al., 2024). These approximations provide a recursive solution to approximating the desirable transfer functions providing computational benefits. Most graph filters allow for a distributed implementation given the use of graph shifts, which can be seen as a result of a diffusion process where only connected nodes may share information (Coutino et al., 2019). This feature is particularly interesting in the field of sensor networks where edge and fog computing approaches can be employed to implement a specific data-driven model.

The notion of bandlimited signals has been widely used to develop signal reconstruction models as well as sensor sampling schemes (Huang et al., 2020). We can define the set of K-bandlimited graph signals as $\mathcal{K} = \{\mathbf{x}\colon \hat{\mathbf{x}}_i = 0 \text{ for } K \leq i \leq N\}$. Many reconstruction algorithms rely on the assumption that a given graph signal has a sparse representation in the spectral domain. Besides, sensor sampling selection schemes have been developed to find the subset of sensor measurements that produce the lowest signal reconstruction error (Rusu and Thompson, 2017).

To conclude this section, we delve into the graph convolution operation, which plays an important role in graph convolutional neural networks (Section 3.3). It is well-known that the graph signal convolution $\mathbf{x} * \mathbf{f}$ is equivalent to the multiplication in the spectral domain (Stanković et al., 2020b):

$$\mathbf{x} * \mathbf{f} = \mathbf{U}\left((\mathbf{U}^{-1}\mathbf{x}) \odot (\mathbf{U}^{-1}\mathbf{f})\right) \tag{12}$$

where $\mathbf{x} \in \mathbb{R}^N$ and $\mathbf{f} \in \mathbb{R}^N$ are both graph signals and $\odot$ corresponds to the Hadamard product. Thus, the convolution is equivalent to the multiplication of the spectral components by a convolution signal $\mathbf{f}$. Recalling the previous Eq. (12) we can observe how a graph filter in the spectral domain can be seen as a convolution operation:

$$\mathbf{x} * g = \mathbf{U}g(\boldsymbol{\Lambda})\mathbf{U}^{-1}\mathbf{x} = \mathbf{U}\begin{bmatrix} g(\lambda_1) & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & g(\lambda_N) \end{bmatrix}\mathbf{U}^{-1}\mathbf{x} \tag{13}$$

Thus, this resembles a graph filter where the polynomials of $\mathbf{L}$ and $\boldsymbol{\Lambda}$ are used to provide good localization properties in the vertex domain. Moreover, approximations such as the Chebyshev polynomials can be used to provide computational benefits (Defferrard et al., 2016). This aspect will be further described in Section 3.3, where spectral GNNs are introduced.

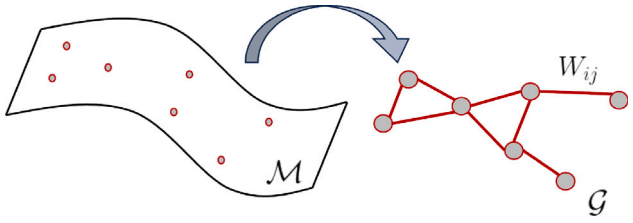

Fig. 9. Representation of data points that lie in a manifold $\mathcal{M}$ and their representation in a graph form $\mathcal{G}$.

3.2. Machine learning over graphs

The use of ML on data residing in a manifold represented by a graph has been in use for years (Song et al., 2022; Subramanya and Talukdar, 2022; Belkin and Niyogi, 2004). In the field of graph-based semi-supervised learning, data is assumed to reside on a manifold, with data points being related to each other, so that models can input such structured information to assist ML models. Other examples of ML tasks include the inference of node embeddings from graph data (Abu-El-Haija et al., 2018). In this context, instead of working directly with graph-structured data, node embeddings are obtained and classic ML models are fed with these node representations.

In this section, we delve into the main uses of ML on graphs in sensor networks, which are the development of graph-regularized models and the interpretability they provide. Although many tasks and techniques could be included in graph ML, e.g., obtaining metrics, graph representation learning, node embeddings, or community detection, in this paper we focus on its use mainly through regularization and kernels in predictive models given its potential applicability to data quality in sensor networks.

3.2.1. Graph-regularized ML

In many cases, it is assumed that the data resides in a non-Euclidean space, such as a smooth manifold or a Riemannian manifold (Belkin and Niyogi, 2004; Zhou and Belkin, 2014). Hence, a graph can be used to describe the similarities of the data in such a manifold. Graph ML models aim to take advantage of a graph that describes the relationship between the data. This concept has been widely used in graph semi-supervised learning, regularization, and kernels over graphs. Thus, a manifold $\mathcal{M}$ is approximated via a graph $\mathcal{G}$, Fig. 9.

The Laplace–Beltrami operator of functionals on Riemannian manifolds is widely used and the graph Laplacian operator can be seen as its discrete approximation when the number of data points tends to infinity (Hu et al., 2021; Burago et al., 2015; Sandryhaila and Moura, 2013a). Then, the smoothness of a function over a graph is represented by the quadratic form $\mathbf{x}^\mathsf{T}\mathbf{L}\mathbf{x}$, which is akin to the graph signal smoothness (Dong et al., 2019; Belkin and Niyogi, 2004). Now, we can illustrate the use of graph-based regularization in a regression task, independently of whether it is supervised or semi-supervised, as finding the map $f\colon \mathbb{R}^M \to \mathbb{R}^N$ where $N > M \geq 1$:

$$\min_f \text{Loss}\left(\mathbf{y}, \mathbf{m} \odot f(\mathbf{x})\right) + \gamma \cdot R_{\mathcal{G}}(f) \tag{14}$$

where $\mathbf{x} \in \mathbb{R}^M$, $\mathbf{y} \in \mathbb{R}^M$, $\mathbf{m} \in \mathbb{B}^N$ is a selector vector that selects the observed entries $\mathbf{x}$ of the graph signal $f(\mathbf{x})$, Loss$(\cdot)$ is a loss function, $\gamma \in \mathbb{R}$ is an hyperparameter promoting the regularization, and $R_{\mathcal{G}}(f)$ is a graph-based regularizing function. An example of regularization is the well-known Tikhonov regularization for graphs where the estimated graph signal $f(\mathbf{x})$ is forced to be smooth with respect to the graph $\mathcal{G}$, the loss function is set to the residual sum of squares and the regularization term is set to the Laplacian quadratic form (Yang et al., 2021):

$$\min_f \underbrace{\|\mathbf{x} - \mathbf{m} \odot f(\mathbf{x})\|_2^2}_{\text{Loss}(\mathbf{x}, f(\mathbf{x}))} + \gamma \cdot \underbrace{f(\mathbf{x})^\mathsf{T}\mathbf{L}f(\mathbf{x})}_{R_{\mathcal{G}}(f)} \tag{15}$$

This is an example of a transductive method, where for a given graph signal with some observed values, the signal is reconstructed taking into account the resulting signal smoothness. This formulation has been widely used for the regression of graph signals in scenarios such as image processing and sensor networks (Belkin and Niyogi, 2004; Venkitaraman et al., 2019; Pilavcı et al., 2021). Similarly, in the context of supervised ML and semi-supervised ML, the norm in the Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$ associated to a graph $\mathcal{G}$ has been used in Laplacian regularized least squares and in Laplacian support vector machines (Melacci and Belkin, 2011), $R_{\mathcal{G}}(f) \equiv \|f\|^{\mathcal{G}}_{\mathcal{H}}$.

More remarkably, the graph counterpart of RKHS has been used in graph ML (Romero et al., 2016; Jian and Tay, 2023; Venkitaraman et al., 2019; Sahbi, 2021). Graph kernels play an important role in ML for comparing graphs in tasks such as graph classification and sub-graph matching in fields such as biology (Borgwardt et al., 2020; Vishwanathan et al., 2010). In sensor networks, however, we are more interested in graph kernels that compare the relationships between nodes in a graph, $k\colon \mathcal{V} \times \mathcal{V} \to \mathbb{R}$. In this sense, we can define a graph kernel matrix $\mathbf{K}_{\mathcal{G}}$ that depicts the similarities between graph nodes as (Romero et al., 2016):

$$\mathbf{K}_{\mathcal{G}} = r^{\dagger}(\mathbf{L}) = \mathbf{U} r^{\dagger}(\boldsymbol{\Lambda}) \mathbf{U}^{-1} \tag{16}$$

where $r\colon \mathbb{R} \to \mathbb{R}_+$ is a non-negative map and $\dagger$ denotes the pseudoinverse. Similarity functions such as graph diffusion or random-walk kernels have been used for graphs. This concept led to the development of graph kernel-based regression where the RKHS, $\mathcal{H}$, is defined for graph signals and its norm is used for regularization (Romero et al., 2016):

$$R_{\mathcal{G}}(f) = \|f\|^2_{\mathcal{H}} = \langle f, f \rangle_{\mathcal{H}} = \boldsymbol{\alpha}^{\mathsf{T}} \mathbf{K}_{\mathcal{G}} \boldsymbol{\alpha} \tag{17}$$

where $\boldsymbol{\alpha} \in \mathbb{R}^N$ is the set of learnable parameters. Then, from the representer theorem we can derive the estimate for the value of the signal at the different nodes (Romero et al., 2016). This concept has been used for kernel-based graph regression as well as Gaussian processes over graphs, kernel-kriged Kalman filters over graphs, and the inference of functions over graphs, using both graph kernels and kernels agnostic to the graph (Venkitaraman et al., 2020; Ioannidis et al., 2018; Romero et al., 2016; Smola and Kondor, 2003).

Finally, many other applications have benefited from graph kernels, e.g., the factorization of matrices by introducing prior information as vertex-based graph kernels (Pal and Jenamani, 2018). Other examples include the introduction of graph kernels for nonnegative matrix factorization (Li et al., 2023a), or the introduction of graph-regularizers in concept factorization techniques (Mu et al., 2023).

3.2.2. Interpretable ML

The interpretability of ML and artificial intelligence models has been the focus of research in recent years given the increasing development and adoption of highly complex models such as neural networks (commonly known as black-box models). The interpretability of a ML model aims to reason why a prediction is made and to give further insights into the performance and functioning of a model (Molnar, 2020; Kök et al., 2023).

Interpretable ML promotes techniques that are interpretable, such as generalized linear regression, generalized additive models, and decision trees among others (Du et al., 2019). Among the most common approaches to interpretability are; *(i)* model-specific interpretability, i.e., interpreting the inferred parameters, such as the coefficients of a linear regression, *(ii)* sparsity-promoting models to force feature selection and enhance the interpretability of the parameters, and *(iii)* model-agnostic interpretability, i.e., providing black-box models with tools to explain the results, such as neural network unrolling, feature visualization, latent embedding inspection, or results inspection.

Signal processing and ML over graphs have been shown to enhance the interpretability of models (Dong et al., 2020). First, a graph provides an explicit structure that indicates the relationships between the sensors as well as the importance of the relationships. Thus, the graph shift matrix defines a node's neighborhood $\mathcal{N}(i) = \{j\colon j \in \mathcal{V} \wedge S_{ij} \neq 0\}$, i.e., the set of influential nodes for a given node. In this line, in graph attention networks (GAT) the attention mechanism defines the importance of the nodes' features (or hidden features) in a similar manner as the graph shift matrix but allows attention at different levels, e.g., different layers and multi-head attention mechanism. The graph learning field seeks to identify the network structure, i.e., relationships between nodes, by promoting smoothness and frequency-domain constraints, therefore, improving the interpretability of the resulting graph. Furthermore, GSP has been used to enhance the interpretability of classic neural networks and graph neural networks by imposing graph-based regularization in hidden layers, forcing the hidden features to adapt to a given graph (Tong et al., 2020). In this line, graph filters and graph signal denoising techniques provide an interpretation in the frequency domain, enhancing the interpretability of the results (Dong et al., 2020). In fact, GNNs have been interpreted as the result of signal denoising and filtering processes (Chen et al., 2021b). Finally, it is worth noting that kernels over graphs and the use of kernelized graph methods also provide an interpretable alternative since the kernel map can denote similarities between nodes and similarities between graphs. The latter concept has been employed in the development of interpretable GNN using graph kernels (Feng et al., 2022b).

To sum up, a graph-based approach for sensor networks can provide insights into the relationships between sensors, just as the graph can be used to introduce prior information. Furthermore, depending on the application of the sensor network, if decision-making processes are to be fed with the network data it is important to be able to interpret the results. For instance, in the case of a network that monitors possible chemical leaks, it is important to be able to interpret the reasoning behind a detection as well as the leakage pattern using the graph.

3.3. Graph neural networks

The major limitation of well-known neural networks such as convolutional neural networks (CNNs) is their intrinsic application on data defined in regular grids such as images, tabular, or structured data. GNNs appeared as the translation of neural networks to data with irregular support such as graphs (Kipf and Welling, 2016a; Defferrard et al., 2016). In this sense, most of the applications tackled by classic neural networks can be applied to GNNs by making use of the graph topology and the operators defined over it. For instance, spectral convolutional GNNs made use of the convolution operator to implement CNNs over data residing on graphs. Another related field is geometric deep learning, where broader geometric properties are taken into account, and tailored layers and techniques are applied to it, but, we do not cover this perspective in this paper.[3]

More precisely, we can broadly classify GNNs into Wu et al. (2020); *(i)* graph convolutional neural networks (GCNs), *(ii)* graph autoencoders (GAEs), and *(iii)* graph recurrent neural networks. Spatiotemporal GNNs are also frequently included in this classification since they incorporate different building modules and may have tailored architectures. In this review, we will mention their main components without going into detail. More precisely, first, we will describe the building blocks of GNNs; *(i)* propagation, *(ii)* sampling, *(iii)* pooling and readout functions. Then, we will describe the main GNN architectures listed above, placing special emphasis on GCNs and their components given that these are widely used and can be used to implement other GNN architectures such as autoencoders. In the next sections, we explain the core elements of the different GNNs, and in Section 4 we review well-known architectures used for specific sensor network applications.

[3] For more information about geometric deep learning refer to Bronstein et al. (2017).

3.3.1. GNNs: Building modules

In this section, we explain the main components that allow the implementation of a broad range of architectures for different GNNs. Among the different components we find those defined in Zhou et al. (2020):

- **Propagation Modules**: given that the data resides in a graph topology, propagation functions that propagate and process the values between different graph nodes are required. For instance, as we will explain in the next section, convolutional GNNs make use of convolution operators using spectral and spatial strategies to share and aggregate information between different graph nodes. Other propagation operators include recurrent operators such as gated recurrent units.
- **Sampling Modules**: are usually included in the propagation modules so that only relevant nodes or nodes' neighborhoods participate in the processing. This is especially interesting for large and highly dense graphs. Examples include the sub-sampling of nodes for each neighborhood, the sampling of edges, or even the sampling of sub-graphs via graph spectral clustering (Chiang et al., 2019).
- **Pooling Modules**: the goal of a graph pooling layer is to obtain coarser representations of the features or graphs. Sub-graphs can be identified and nodes and edges can be aggregated. This idea is related to the down-sampling notion of a signal where some coarser nodes are generated to represent a higher dimensional graph signal (Liu et al., 2023b). The most simple techniques for pooling are the simple $sum/mean/max$ functions (Henaff et al., 2015):

$$\mathbf{h}^{(k)} = pool_{\mathcal{G}}\left(\mathbf{h}_1^{(k)}, \dots, \mathbf{h}_{C_k}^{(k)}\right) \tag{18}$$

where $\mathbf{h}_i^{(k)}$ represents the hidden features at the kth layer and the ith channel. Some other more sophisticated techniques, such as hierarchical pooling strategies, rearrange the graph nodes and create a new reduced set of nodes and edges representing the original ones (Bianchi et al., 2020). Some examples include graph coarsening and eigenpooling (Ma et al., 2019). These techniques play an important role in scenarios where the output of the GNN is at graph level or edge level, meaning that some reduced graph representations might be required. Readout functions can be used to summarize final or intermediate representations of a graph so that new features can be obtained, e.g., aggregating hidden representations. These are particularly important in GNNs where the output is expressed at graph level, e.g., graph classification, so that the readout function produces a metric summarizing the entire graph (Wu et al., 2020). Moreover, readout functions can be used to produce node embeddings and aggregations so that classical, e.g. fully-connected or convolutional layers, can be used within a GNN.

3.3.2. Convolutional GNNs

Convolutional GNNs are commonly classified into Wu et al. (2020) and Zhou et al. (2020); *(a)* spectral GNNs and *(b)* spatial GNNs. Spectral convolutional GNNs make use of the definition of graph signal convolution operator (Section 3.1.3) to implement the classic convolution operation, while spatial convolutional GNNs make use of the graph structure to define nodes' neighborhoods and implement the convolution.

Spectral convolutional GNNs: make use of the graph convolution operator $\mathbf{x} * \mathbf{f}$ to define the convolution layers (Kipf and Welling, 2016a; Defferrard et al., 2016). In a general manner, we can define a spectral graph convolution layer as:

$$\mathbf{h}_j^{(k)} = \sigma\left(agg_{\ i=1,\dots,c_{k-1}}\left(\mathbf{h}_i^{(k-1)} * \boldsymbol{\theta}_{ij}^{(k)}\right)\right), \qquad j = 1, \dots, c_k \tag{19}$$

where $\mathbf{h}_{c_k}^{(k)} \in \mathbb{R}^N$ is the c_l (channel) hidden state at layer k, $\sigma(\cdot)$ is the activation function, $c_k \in \mathbb{N}^+$ is the number of channels in layer k, $agg(\cdot)$ is an aggregation function used to aggregate the different filtered versions of graph signal $\mathbf{h}_i^{(k-1)} \in \mathbb{R}^N$ (e.g., sum or average), and $\boldsymbol{\theta}_{ij}^{(k)} \in \mathbb{R}^{K_{ij}}$ is the convolution function associated with a set of learnable parameters. Different types of networks and convolutions assume a different parametrization of the learnable parameters $\boldsymbol{\theta}$. For instance, the GCN (Kipf and Welling, 2016a) and Chebnet (Defferrard et al., 2016) use the Chebyshev polynomials to approximate the filtering, the CayleyNet (Levie et al., 2018) uses the Cayley polynomials to approximate the filtering, the Graph Wavelet Neural Network (GWNN) (Xu et al., 2019) uses wavelets to approximate the convolution operation, and the graph diffusion neural network (GDNN) (Gasteiger et al., 2019) uses a graph diffusion convolution to implement the message passing operation, combining strengths of spatial and spectral methods. For instance, the implementation of the spectral graph convolution using the truncated expansion of the Chebyshev polynomials results in (Defferrard et al., 2016):

$$\mathbf{x} * \boldsymbol{\theta} = \sum_{k=0}^{K-1} \theta_k T_k(\mathbf{L})\mathbf{x} \tag{20}$$

where $\boldsymbol{\theta} \in \mathbb{R}^K$ are the polynomials coefficients and $T_k(\mathbf{L})$ corresponds to the recursive implementation of the truncated Chebyshev polynomials of $\mathbf{L}$ (Defferrard et al., 2016). Fig. 10 depicts a classic architecture for a convolutional GNN where different pooling modules and layers can be included.

Spatial convolutional GNNs: in this case, the spatial notion in a graph $\mathcal{G}$ is used, where the graph structure explicitly tells which nodes are related (node neighborhood) and to what extent (Zhang et al., 2019; Danel et al., 2020). Among the most representative spatial graph convolutional neural networks we find propagation-based models such as the diffusion-based graph convolutional neural network (DCNN) (Atwood and Towsley, 2016), the GraphSAGE (Hamilton et al., 2017) or generalistic frameworks such as message-passing neural networks (MPNNs) (Gilmer et al., 2017). Nonetheless, we can define a general spatial graph convolutional layer as:

$$\begin{aligned}\mathbf{a}^{(k-1)}(n) &= agg_{\forall i}\left([\mathbf{h}_i^{(k-1)}(n) \parallel \mathbf{h}_i^{(k-1)}(\mathcal{N}(n))], \boldsymbol{\theta}_i^{agg,(k)}\right), \\ \mathbf{h}_j^{(k)}(n) &= f\left(\mathbf{h}_j^{(k-1)}(n), \mathbf{a}^{(k-1)}(n), \boldsymbol{\theta}_j^{f,(k)}\right) \quad , j = 1, \dots, c_k\end{aligned} \tag{21}$$

where $f(\cdot)$ is a function that combines the value at node n with a function of the vicinity, $agg_{\forall i}(\cdot)$ is a function that aggregates the information of neighboring nodes of $\mathcal{N}(n)$, and $\parallel$ denotes a concatenation operation. Function f combines the aggregated spatial information at node n ($\mathbf{a}^{(k-1)}(n)$) with the previous hidden state $\mathbf{h}^{k-1}(n)$ at node n. Both agg and f can contain a set of learnable parameters $\boldsymbol{\theta}_i^{agg}$ and $\boldsymbol{\theta}_j^{f}$. This framework provides great generalization capabilities allowing the definition of aggregation and combination functions according to the data. The GAT (Velickovic et al., 2017) make use of the node spatial notion and graph structure to define the graph-based analogs of attention mechanisms.

3.3.3. Autoencoder GNNs

GAEs are widely used in applications such as outlier detection and missing value imputation. GAEs are trained to reconstruct the input signal, and in the process, a latent representation of the data is learned. To force the neural network to learn a latent representation, the dimensions of the hidden features of the encoder are decreased to produce a bottleneck effect, in what is called an undercomplete autoencoder, or the dimensions are increased, in what is called an overcomplete autoencoder. Hence, this idea is related to representation learning, where latent representations of the data are learned, either by obtaining a compressed version of the data or an overexpressed version. Variational graph autoencoders (VGAE) (Kipf and Welling, 2016b) appeared to perform edge-level prediction tasks by making use of graph convolution layers to produce the latent representation of the data. In fact, graph convolution layers are the building blocks of GAEs, however, other layers can be used (Salha et al., 2019). Graph variational

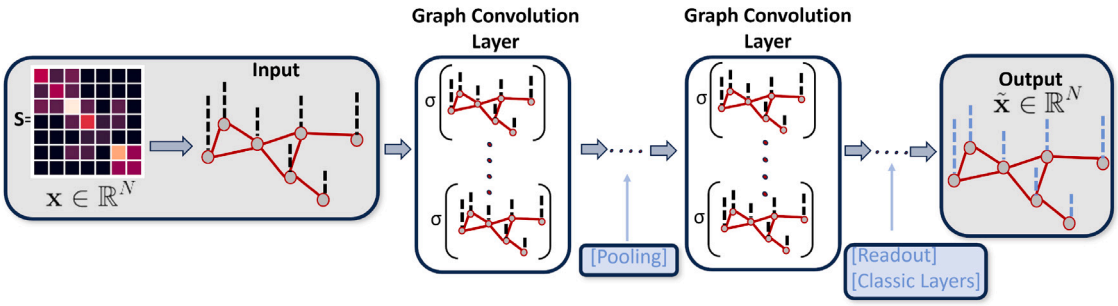

Fig. 10. Example of a GCN architecture at the node level, where a graph signal $\mathbf{x}$ is reconstructed $\tilde{\mathbf{x}}$. The inputs are the graph signal and the graph shift matrix $\mathbf{S}$, and $\sigma(\cdot)$ denotes an activation function. A GNN architecture can include graph convolution and pooling layers, as well as readout layers and classic layers, e.g., fully-connected or convolutional layers.

autoencoders (GVAE) (Simonovsky and Komodakis, 2018) also make use of a GCN for the encoder. Graph autoencoders typically use a graph convolution approach to produce the latent features and the decoder part can use some well-known neural network layers to produce the output. An example is the marginalized graph autoencoder (Wang et al., 2017) that combines the use of graph convolutional layers with spectral clustering. Adversarially regularized graph autoencoders (ARGA) (Pan et al., 2018) make use of a graph convolution-based encoder and a graph embedding-based decoder to perform link prediction. Dirichlet graph variational autoencoders (DGVAE) (Li et al., 2020b) use Dirichlet priors and spectral clustering to enhance variational autoencoders. All in all, GAEs can be implemented by means of the layers seen so far, combining ideas from representation learning and dimensionality reduction to perform tasks at different levels, e.g., signal reconstruction, and link prediction.

3.3.4. Recurrent GNNs

Recurrent GNNs usually combine GCNs with recurrent modules to take into account the temporal patterns of the data (Liu and Zhou, 2022). For instance, graph convolutional recurrent neural networks (GCRNs) put LSTM and the ChebNet together. Diffusion convolutional recurrent neural networks (DCRNNs) (Li et al., 2018) combine diffusion graph convolutional layers with gated recurrent units. Gated graph neural networks (GGNNs) (Li et al., 2016) use gated recurrent units with spatial convolutions to implement the recurrence (Wu et al., 2020):

$$\mathbf{h}^{(k)}(n) = \mathbf{GRU}\left(\mathbf{h}^{(k-1)}(n), agg_{u\in\mathcal{N}(n)}\left(\mathbf{h}^{(k-1)}(u), \boldsymbol{\theta}\right)\right) \tag{22}$$

where the hidden representation of node n is a gated recurrent unit applied to the previous state of the nth node ($\mathbf{h}^{k-1}(n)$) and a combination of the neighbors' previous representations $agg_{u\in\mathcal{N}(n)}(\mathbf{h}^{(k-1)}(u), \boldsymbol{\theta})$. $\boldsymbol{\theta}$ are learnable parameters. These are just some examples of recurrent neural networks and how these are implemented by means of convolution operations together with classic recurrent units.

Spatiotemporal graph neural networks combine both convolutional and recurrent layers to learn spatial and temporal dependencies, which can be encoded by time-varying graphs and signals. For instance, graph convolutions can be used to model spatial dependencies, and gated units or classic convolutional layers can be used to model temporal dependencies. Common applications include forecasting techniques (Li and Zhu, 2021; Bui et al., 2022).

3.3.5. Generative GNNs

Generative models have attracted attention in recent years. Specifically, generative models can generate new synthetic measurements, or more complex structures such as graphs and sub-graphs. There are two main classes of models that are the most widely used, the variational autoencoders and the generative adversarial networks (GANs). In the context of graph variational autoencoders, models have been developed that make use of GCNs as encoders and learn a latent generative model $q(h)$ (where h is the latent representation of the data) that allows for generating new instances of the graph shift matrix or graph signals (Kipf and Welling, 2016b; Ahn and Kim, 2021). For example, Do et al. (2020) use a VGAE consisting of an encoder formed by stacked layers of GCNs, a latent model parametrized as a Gaussian distribution, and a decoder composed of stacked GCNs to complete air quality measurements. Another successful example of generative models are GANs in which a sample generator g competes with a discriminator whose goal is to distinguish real instances from generated instances in a zero-sum game (Pan et al., 2018; Wang et al., 2018). For example, VGAEs can be used as generators to generate synthetic samples or graphs (Pan et al., 2018) or more customized techniques can be developed to generate possible graph shift matrices (Wang et al., 2018).

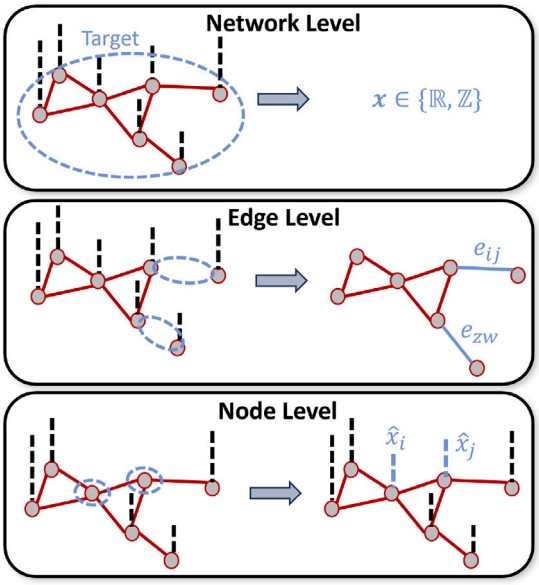

Fig. 11. Different task levels for graph-based techniques in monitoring sensor networks; network level tasks, edge level tasks, and node level tasks.

Throughout this section, we have reviewed the foundations of GSP, ML over graphs and GNNs. In the next section, we review articles that apply many of the fundamentals we have seen to improve data quality in the context of monitoring sensor networks using graph models.

4. Graph-based data quality applications

As we have reviewed in previous sections, many graph-based techniques allow the modeling of sensor network measurements (Dong et al., 2023). There is a wide variety of applications where graphs can

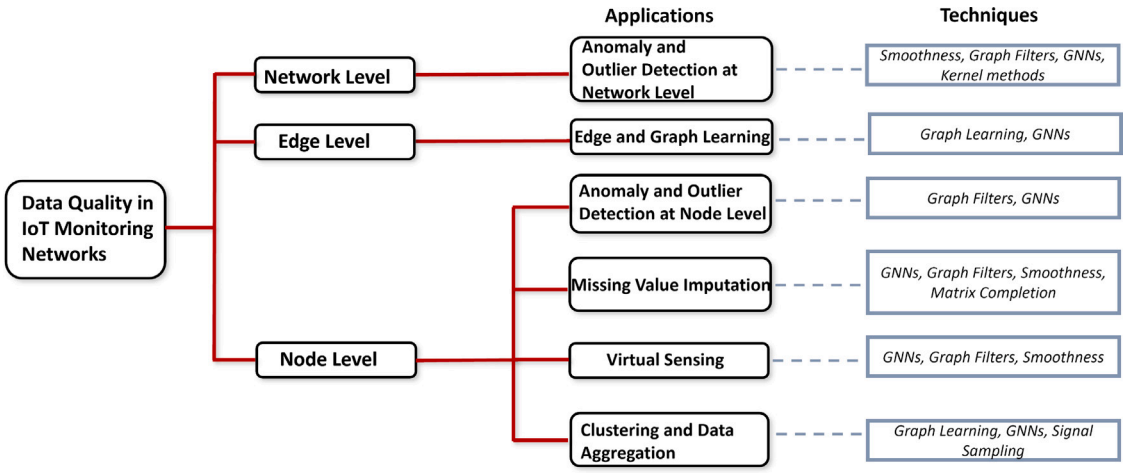

Fig. 12. Proposed taxonomy for the different data quality-enhancing applications that can be carried out in a monitoring sensor network and the techniques that can be used.

be leveraged to perform data quality-enhancing tasks, such as outlier detection or virtual sensing among others. Before delving into specific use cases, let us roughly classify the tasks according to their scope:

- *Network level*: the outcome is obtained at the network (graph) level, i.e., the measurements at the different sensors and the graph topology are fed into a model to summarize the behavior of the collected measurements. For instance, detecting whether something is wrong within the network at a time instant, detecting events, or obtaining global metrics.
- *Edge level*: in this case, the targets are the graph edges, e.g., link prediction. The goal is to predict or obtain a set of edges from the sensors' measurements. For instance, this approach can be used to estimate new connections between sensors, e.g., recently joined sensors, or the entire graph.
- *Node (Sensor) level*: this is the most common case, where the target is the prediction of a feature at the sensor level, e.g., missing value imputation at different sensors using the sensor network measurements.

Fig. 11 represents the different task levels. Moreover, Fig. 12 presents the data quality-enhancement task taxonomy described in this section, as well as the theoretical aspects used for each one of the applications — more details are given in the following sections. At the *network level*, we find tasks such as the *detection of outliers and anomalies* which can be critical in assessing data consistency and coherence. At the *edge level*, we find tasks such as *edge prediction and graph learning* which might help in identifying correct graph structures and further applications to improve data accuracy. At last, at the *node level*, we find applications such as *outlier detection* to improve data consistency and coherence by identifying sensor measurement deviations, *missing value imputation* to ensure data completeness, *virtual sensing* to tackle different data quality dimensions such as accuracy or completeness, or *clustering* which can be useful in grouping the data and improving data accuracy in further application. All these applications can be used to tackle data quality issues to produce continuous, complete, and accurate measurements (Ferrer-Cid et al., 2024a).

4.1. Network level

Let us define a network level task as finding the map $f_{\mathcal{G}}\colon \mathbb{R}^{N\times T}\to\mathbb{R}$ such that $y = f_{\mathcal{G}}(\mathbf{X})$ where $\mathbf{X}\in\mathbb{R}^{N\times T}$ is a graph signal ($T = 1$) or a set of graph signals ($T > 1$). The target variable is y, which can be a categorical variable (e.g., classification or outlier detection task) or a scalar value representing a metric for the graph.

4.1.1. Anomaly and outlier detection at network level

In the case of outlier and anomaly detection at the network level, we are interested in classifying a graph signal or a set of graph signals, i.e., $y\in\{1,\dots C\}$ where $C\in\mathbb{N}^+$ is the number of possible classes. For instance, the case of detecting whether the network measurements in a time instant correspond to an anomalous graph signal or not ($y\in\{0,1\}$). Hence, this approach does not provide localization capabilities, i.e., identifying the suspicious sensor or measurements that are causing the anomaly. Different ML approaches fit this setting; supervised ML, unsupervised learning, and semi-supervised learning. Supervised and semi-supervised models are limited in this field since labeled data about faulty sensors and abnormal measurements is difficult to obtain. The most common approach is the unsupervised learning setting where no labeled data is used and the data distribution together with the graph topology and a model are used. Ideally, data points, e.g., graph signals, that deviate from the distribution of the majority of observed graph signals are assumed to be anomalous.

GSP metrics such as total variation or graph signal smoothness have been used to identify anomalous graph signals, e.g., $S(\mathbf{S},\mathbf{x}) > TH$, where $TH\in\mathbb{R}$ is a threshold value indicating the graph signals presenting discrepancies between connected sensors which are prone to be anomalous (Gopalakrishnan et al., 2019; Chen et al., 2015). Other GSP methods use the magnitude of the GDFT high-frequencies, the spectral graph wavelet transform, or the GDFT representation of a graph filter output to detect anomalous signals, e.g., $\|\hat{\mathbf{x}}_{\text{high}}\| > TH$, Egilmez and Ortega (2014), Lewenfus et al. (2019), Xiao et al. (2020a) and Sandryhaila and Moura (2014b). For instance, Xiao et al. (2020a) detected anomalies in a temperature monitoring IoT network by analyzing the frequency response of a graph filter.

Recently, Su et al. (2024) have developed a graph-frequency domain filter which together with a Kalman filter is able to filter out anomalous graph signals in industrial pipe networks. Residual-based models have also been used to detect anomalies, where a graph signal reconstruction model, e.g., graph filter or GNN, is fitted with the data and large residuals' norm depicts a possible outlier, $\|\mathbf{x}-\mathbf{f}_{\mathcal{G}}(\mathbf{x})\| > TH$. Similarly, GNNs and GCNs have also been used as autoencoders to detect anomalous graph signals focusing on both signal reconstruction residuals and the reconstructed signal spectrum (Zhou et al., 2023; Tang et al., 2022). Moreover, spatiotemporal GNNs have been used to detect anomaly signals in multivariate time series, including anomalies in water monitoring sensor networks (Chen et al., 2021a). Lin et al. (2022) propose a context-aware graph autoencoder to detect anomalous air pollution events in an air quality monitoring network in Beijing. Feng et al. (2022a) propose a GAE based on graph convolutional layers to reconstruct the graph edges and graph signals and compute the anomaly score based on the loss function and the kernel density

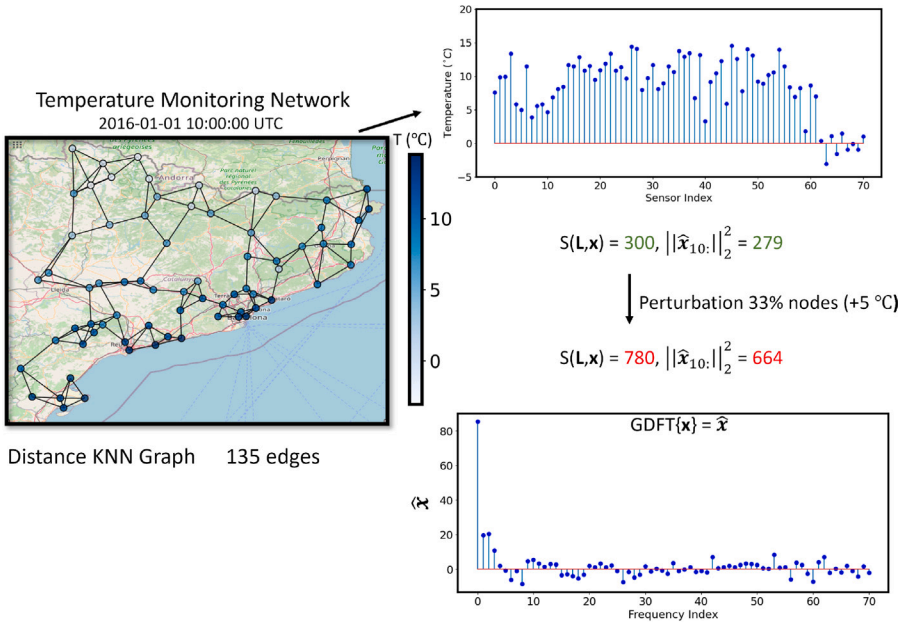

Fig. 13. Example of a temperature monitoring network in the area of Catalonia, Spain, with a total of 71 nodes. The signal smoothness, $S(\mathbf{L}, \mathbf{x})$, and the norm of the GDFT coefficients associated to high-frequency components, $\|\hat{\mathbf{x}}_{10:}\|$, are used to detect anomalous measurements.

estimation of the distribution of scores during the training. Xu et al. (2021) propose an edge-based GCN to detect anomalous electricity consumption records in smart meter networks in a graph classification supervised manner.

Graph regularization can also be coupled with matrix factorization techniques, some examples include nonnegative matrix factorization or graph-regularized sparse representation for anomaly detection in other fields (Huang et al., 2018; Li et al., 2021b).

Real-world use case: Fig. 13 shows an outlier detection example for a temperature monitoring network. This IoT monitoring network consists of 71 sensors deployed in the area of Catalonia, Spain. First, a distance-based K-nearest neighbors graph, Section 3.1.1, has been constructed, resulting in a Laplacian matrix $\mathbf{L}$ with 135 edges. Then a temperature signal has been taken at a specific time and the smoothness ($S(\mathbf{L}, \mathbf{x}) = 300$) and magnitude of the higher-order coefficients of the GDFT ($\|\hat{\mathbf{x}}\|_2^2 = 279$) have been computed. Then, 33% of the nodes were perturbed with a shift of +5 °C to simulate an anomalous signal. As it can be seen, both the smoothness and the magnitude of the higher-order GDFT coefficients can be used to detect the anomalous signal ($S(\mathbf{L}, \mathbf{x}) = 780$ and $\|\hat{\mathbf{x}}\|_2^2 = 664$). Other techniques could include the use of graph filters or GNNs and the subsequent evaluation of the norm of the residuals or the GDFT coefficients followed by a thresholding procedure. In similar real-world use cases, a model based on GDFT and a GCN were compared when applied to temperature sensor networks (Xiao et al., 2020a). Similarly, the GAE proposed by Feng et al. (2022a) was compared with VGAE in an industrial IoT network.

All in all, residual-based models are used to detect anomalies and outliers in an unsupervised manner, the signal reconstruction can be implemented in different ways, e.g., convex optimization, graph filters, GNN, and the anomaly score is computed based on some norm or distance. Graph filters can provide a scalable, interpretable, and convex alternative to highly complex GNNs. Nevertheless, depending on the data availability, GNNs can provide superior performance due to their ability to model multivariate time series, including temporal components and complex surfaces. In any case, as we will see in Section 5.2.1, GNNs and GSP tools can provide benefits in terms of model generalization and domain transferability, so easing the training of models and dealing with limited data scenarios. Table 2 lists some graph-based models as well as their use cases.

4.2. Edge level

Let us define an edge level task as discovering a set of graph edges $e \subseteq \mathcal{E}$ or learning a graph shift matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ given a set of measurements $\mathbf{X} \in \mathbb{R}^{N \times T}$ or some prior assumptions. In this section, we place special emphasis on the use of graph learning, see Section 3.1.1, to learn the relationships between sensors in a network, thus revealing patterns and allowing the use of clustering or other graph-based techniques. As mentioned in Section 3.1.1, several graph learning techniques can be used for graph structure inferring (Dong et al., 2019; Mateos et al., 2019). In most cases, the graph learning problem is formulated as an optimization problem where the graph shift matrix is restricted to the set of valid graph matrices. For instance, Jabłoński (2017) applies a smoothness-based graph learning model to discover the relationships between reference stations of tropospheric ozone (O_3) to unveil the implicit relationships and apply clustering algorithms. Similarly, Mei and Moura (2016) apply casual graph learning to infer the temperature diffusion pattern in the United States. Ferrer-Cid et al. (2021) discuss the use of different graph learning algorithms for representing air quality monitoring sensor networks. Likewise, Allka et al. (2024) perform graph learning over a sensor network to discover the relationships between the sensors and group the data to feed a neural network-based anomaly detector. Cini et al. (2023) develop a sparse graph learning algorithm and applied it to an air quality index sensor network. Jiang et al. (2021) propose an optimization problem to jointly perform graph learning and matrix completion over a temperature sensor network using a smoothness-promoting approach. Similarly, Liu et al. (2019) propose a graph learning model for time-varying graph signals that takes into account the spatiotemporal smoothness of the graph and provides good results for temperature sensor networks. Apart from techniques based on formulating the graph learning problem as an optimization problem using some assumptions such as signal smoothness, there are other approaches based on GNNs, e.g., VGAE, that have been used to infer the topology of the graph or estimate some of the links of the graph as well as to perform a specific task (Zhang et al., 2023c; Dehmamy et al., 2019). Chen et al. (2021a) use a learning policy to reduce the computational costs of learning the graph structure. In the context of precision agriculture, Vyas and Bandyopadhyay (2022) propose a soil moisture prediction task coupled with the identification of a dynamic graph structure.

Table 2
Examples of anomaly and outlier detection graph-based models applied at the network level.

		Reference	Year	Method	Use case
GSP	Smoothness	Chen et al. (2015)	2015	TV	Temperature sensor network
		Gopalakrishnan et al. (2019)	2019		Delays US airports
	Graph filter	Egilmez and Ortega (2014)	2014	High-frequencies graph filter	Synthetic sensor network
		Lewenfus et al. (2019)	2019	Spectral Graph Wavelet Transform (SGWT)	Temperature sensor network
		Sandryhaila and Moura (2014b)	2014	High-frequencies GDFT	
		Su et al. (2024)	2024	Graph-frequency domain Kalman filter	Industrial pipe network
		Xiao et al. (2020a)	2020	GDFT of Nonlinear Polynomial Graph Filter (NPGF)	Temperature sensor network
GNN	GCN & GAE	Chen et al. (2021a)	2021	GCN & Attention mechanism	Water sensor network
		Feng et al. (2022a)	2022	GAE using GCN	Industrial IoT network
		Lin et al. (2022)	2022	Context aware GAE	Air quality sensor network
		Xu et al. (2021)	2021	GCN	Smart meter network

In short, although there are approaches based on optimization and GNNs, for sensor networks it is still very common to establish the network topology from the geodesic distances between sensors. For instance, Do et al. (2020) establish the graph adjacency matrix using the geodesic distances of sensors and locations for air quality sensor networks. Nonetheless, data-driven approaches can learn tailored graphs that better represent the implicit relationship between sensor nodes as well as graphs tailored to suit a specific task or application. These data-driven graphs can improve the performance of further applications, e.g., missing value imputation (Jiang et al., 2021; Ferrer-Cid et al., 2022a). Convex optimization problems can provide an informative alternative to GNNs that learn the graph representation for a specific application.

4.3. Node level

Let us define a node-level task as finding the map $f_{\mathcal{G}}\colon \mathbb{R}^{N_1} \to \mathbb{R}^{N_2}$ such that $\mathbf{y} = f_{\mathcal{G}}(\mathbf{X})$ where $\mathbf{X} \in \mathbb{R}^{N_1 \times T}$ is a graph signal ($T = 1$) or a set of graph signals ($T > 1$) and $\mathbf{y}$ is the target variable, which can be categorical. For instance, binary vector in an outlier detection task where each graph node can be identified as anomalous, or a scalar-valued vector for the prediction of graph signals at different nodes. Value N_1 denotes the number of observed nodes in the graph and value N_2 denotes the number of target nodes in the graph.

4.3.1. Anomaly and outlier detection at node level

Outlier and anomaly detection at the node level is especially interesting in IoT monitoring sensor networks since it localizes the sensor with faulty measurements and this can lead to further replacement or maintenance actions. This is known as outlier/anomaly localization. In this case, the output of the outlier detection process is $\mathbf{y} \in \{0,1\}^N$, where each component y_i indicates whether the ith sensor is presenting an anomaly.

The most common approach for the outlier detection and localization task in monitoring sensor networks is the use of residual-based models. These models train a signal reconstruction model using data with normal patterns and then the reconstruction residuals are used to identify the outliers. Following the GSP framework, linear and nonlinear graph filters have been used in the detection of outliers, $|x_i - f_{\mathcal{G}_i}(\mathbf{x})| > TH$, for air quality and temperature sensor networks (Ferrer-Cid et al., 2022b; Xiao et al., 2020b). Some of these graph filters allow a distributed implementation, e.g., the distributed nonlinear polynomial graph filter (NPGF) (Xiao et al., 2021).

GNNs naturally adapt to this application, allowing the implementation of graph autoencoders using attention mechanisms, convolution, and pooling layers (Ren et al., 2023; Deng and Hooi, 2021). This problem can also be seen as the detection of outliers in multivariate time series, where the time series of the sensor network measurements are fed to a graph convolutional adversarial network (Deng et al., 2022). State-of-the-art techniques include the use of GNNs and graph convolutional autoencoders. For instance, Miele et al. (2022) propose a graph convolutional autoencoder to detect anomalies in wind monitoring sensor networks. Wu et al. (2023) propose the use of spatiotemporal GCN and graph wavelet networks for the detection of malfunctioning fine particulate matter ($PM_{2.5}$) sensors in a large-scale sensor network, providing a centralized and a localized solution. Other works make use of spatiotemporal graphs and recurrent GCN and graph coarsening to detect anomalous sensors using sophisticated anomaly scores (Yang and Yue, 2022). Darvishi et al. (2023) highlight the importance of outlier detection in IoT networks used in the development of digital twins, in particular, they use a deep recurrent GCN to create virtual sensors and use the residuals to detect sensor failures. Han and Woo (2022) propose a GCN that is fed by the representation learned by a sparse autoencoder. Residuals are standardized providing explicit anomaly localization and the maximum score is used as anomaly metric, showing good results for water sensor networks. Besides, other methods focus on obtaining a list of outlying nodes given a set of measurements, e.g., Francisquini et al. (2022) develop a graph filter to detect consistently outlying nodes. Table 3 summarizes different graph-based models for outlier detection at the node level.

All in all, residual-based models are commonly used for anomaly and outlier detection in sensor networks since they allow for the computation of a metric per sensor, allowing the localization of the faulty sensor. The most common techniques include the use of graph filters, signal smoothness, and GNNs. The main advantage of GNNs is their ability to model complex and heterogeneous phenomena, as their architecture can include classical layers for multivariate cases, temporal components, or attention mechanisms. Nevertheless, graph filters provide an interpretable alternative that depending on the data can provide competitive performance, providing good distributed and generalizable approaches, Section 5.2.1. Although not common, graph clustering techniques could also be employed to detect anomalous sensors in a sensor network given that they have been used successfully in other fields (Liu et al., 2021).

4.3.2. Missing value imputation

Sensor networks are prone to losses, either due to sensor, communication or storage failures. Therefore, it is common for measurements to present losses, i.e., at time t, only a subset of sensors $\mathcal{M} \subset \mathcal{V}$ collect measures. Given its importance, missing value imputation has been a very active field of research, leading to the creation of different imputation models. This task is tackled at the node level since the main goal is to infer the graph signal components at different nodes, e.g., imputing the measurements of certain sensors in the network. The imputation of missing values in sensor networks is also referred to as matrix completion, graph signal reconstruction, or multivariate time series imputation or forecasting, among others.

Models based on matrix completion are transductive by nature, use a matrix of spatiotemporal measurements, $\mathbf{X} \in \mathbb{R}^{N \times T}$, with missing values, and their objective is to impute the missing entries of the matrix via the minimization of a specific matrix norm or regularization strategy. GSP-based matrix completion algorithms have been proposed

Table 3
Examples of anomaly and outlier detection graph-based models applied at the node level.

		Reference	Year	Method	Use case
GSP	Graph filter	Ferrer-Cid et al. (2022b)	2022	Volterra Graph Filter Outlier Detection (VGOD)	Air quality sensor network
		Francisquini et al. (2022)	2022	Spectral graph filter community-based detection	Water sensor network
		Xiao et al. (2020b)	2020	NPGF	Temperature sensor network
GNN	GCN & GAT	Darvishi et al. (2023)	2023	Deep recurrent GCN	
		Deng and Hooi (2021)	2021	Graph learning & GAT	Water sensor network
		Han and Woo (2022)	2022	Sparse autoencoder & GCN	
		Wu et al. (2023)	2023	GCN & Graph wavelet networks	Air quality sensor network
		Yang and Yue (2022)	2022	Residual GCN & Latent Spatial-Temporal Graph Modeling (LSTGM)	Water sensor network

Table 4
Examples of graph-based missing value imputation methods at node level.

		Reference	Year	Method	Use Case
GSP	Smoothness	Betancourt et al. (2023)	2023	Random forest & Graph signal smoothing	Air quality sensor network
		Ferrer-Cid et al. (2022a)	2022	Graph learning & signal smoothness	
		Han et al. (2019)	2019	Matrix completion with signal smoothness regularization	Speed monitoring sensor network
		Jiang et al. (2021)	2021	Graph learning & Signal smoothness matrix completion	Air quality sensor network
		Lei et al. (2022)	2022	Kernel signal smoothness matrix factorization	Speed monitoring sensor network
		Mondal et al. (2022)	2022	Sobolev norm matrix completion	Weather monitoring network
		Qiu et al. (2017)	2017	Time-Varying signal smoothness	Air quality sensor network
GNN	GCN	Castro-Correa et al. (2023)	2023	Time GNN - GCN and smoothness regularization	
		Chen et al. (2022)	2022	Adaptive GCN	Air quality sensor network
		Spinelli et al. (2020)	2020	Adversarial graph denoising autoencoder	
	Other GNN	Wu et al. (2021b)	2021	Inductive GNN Kriging (IGNNK)	Precipitation monitoring network
		Zhang et al. (2023a)	2023	Spatiotemporal variational autoencode	Weather monitoring network
		Cini et al. (2022)	2022	GRIN	Smart meter network
		Marisca et al. (2022)	2022	Sparse spatiotemporal attention GNN	Air quality sensor network
		Zhang et al. (2023b)	2023	GLPN	Solar power sensor network

where signal smoothness metrics, e.g., Sobolev smoothness, are optimized (Han et al., 2019; Deng et al., 2021; Mondal et al., 2022). Other frameworks jointly learn the graph and complete the missing data via matrix completion using the $l_{2,2}$ and nuclear norms (Jiang et al., 2021). Besides, matrix factorization techniques have also been used for missing value imputation by using graph kernels to model spatial relationships (Lei et al., 2022). There also exist models that tackle the missing value imputation problem using graph signal reconstruction techniques based on the signal smoothness criterion, with applications in air quality sensor networks (Ferrer-Cid et al., 2022a; Betancourt et al., 2023). For instance, Qiu et al. (2017) impute $PM_{2.5}$ sensor measurements by optimizing the time-varying smoothness of graph signals.

In terms of GNNs, these have been widely used for imputing multivariate time series, which can be interpreted as a set of graph signals collected by sensor networks. The most common approach for matrix completion using GNNs is to train a GNN in the form of an autoencoder so that the reconstruction error over the non-missing entries of the matrix of observations is minimized (Spinelli et al., 2020). It is worth noting that ground-truth values for missing entries are not available, therefore, loss functions can be computed over observed measurements or surrogate objective functions, e.g., artificially introduced missings or forecasting (Cini et al., 2022). Examples of GNN architectures used include gated GNNs (Liu et al., 2020b), GCNs (Chen et al., 2022; Zhang et al., 2023b) or adversarially-trained GCNs (Spinelli et al., 2020). For instance, Chen et al. (2022) develop a bidirectional recurrent GNN based on graph convolutional layers to perform matrix completion and online imputations in environmental data sets. Similarly, Zhang et al. (2023a) propose a spatiotemporal variational autoencoder composed of graph convolution layers and recurrent units to impute missing values of multiple pollutants in environmental data sets. Zhang et al. (2023b) introduce the graph Laplacian pyramid network (GLPN) (Zhang et al., 2023b) which proposes a signal's Dirichlet energy maintenance step to refine the imputation. Other GNNs have been used for missing value imputation like graph recurrent imputation network (GRIN) (Cini et al., 2022), inductive graph neural networks that provide generalizable architectures to graphs of different topologies (Wu et al., 2021b), or GNNs with sparse spatiotemporal attention mechanisms (Marisca et al., 2022). Table 4 summarizes the different missing value imputation methods described above. Even though convex models based on GSP have been proven useful for missing value imputation, GNNs can provide superior performance for large data sets given their ability to model highly complex functions.

Real-world use case: Fig. 14 shows a missing value imputation example for an air quality monitoring network. The IoT monitoring platform measures tropospheric ozone (O_3) at 8 sites in the Barcelona area, Spain. Firstly, a data-driven graph (Laplacian matrix $\mathbf{L}$) is learned from the data matrix $\mathbf{X} \in \mathbb{R}^{N \times T}$, where N is the number of sensors and T the number of available measurements, using the methodology described in Dong et al. (2016). Then, matrix $\mathbf{Y}$ represents the collected network measures for a given time period, where there are some data gaps or missing values. To impute the missing values, a Sobolev smoothness matrix completion approach is used (Mondal et al., 2022). The imputation results in an $R^2 = 0.91$. Other graph-based missing value imputation approaches could be utilized, e.g., other signal smoothness-based criteria or GCNs making use of the learned shift matrix $\mathbf{L}$. Online variants or small data batches could be leveraged to impute missing values in real time. In similar real-world use cases, the adaptive GCN proposed by Chen et al. (2022) was compared with the GRIN for the imputation of $PM_{2.5}$, radiation, and humidity measurements in a network of over 500 sensors.

All in all, graph topologies provide an explicit approach to modeling the spatiotemporal relationships between sensors, which are exploited by graph-based models. It is worth mentioning that the reconstruction of a sensor signal, i.e., the creation of a virtual sensor for that sensor, can be used to impute measurements in case of losses and vice versa, a scenario seen in the next Section 4.3.3. Most effective techniques include matrix completion methods that pose a graph smoothness-based optimization method to complete graph signals. These techniques may face scalability challenges in large-scale sensor networks, e.g., advanced optimization techniques like the alternating direction method of multipliers (ADMM) have been used to ease scalability problems

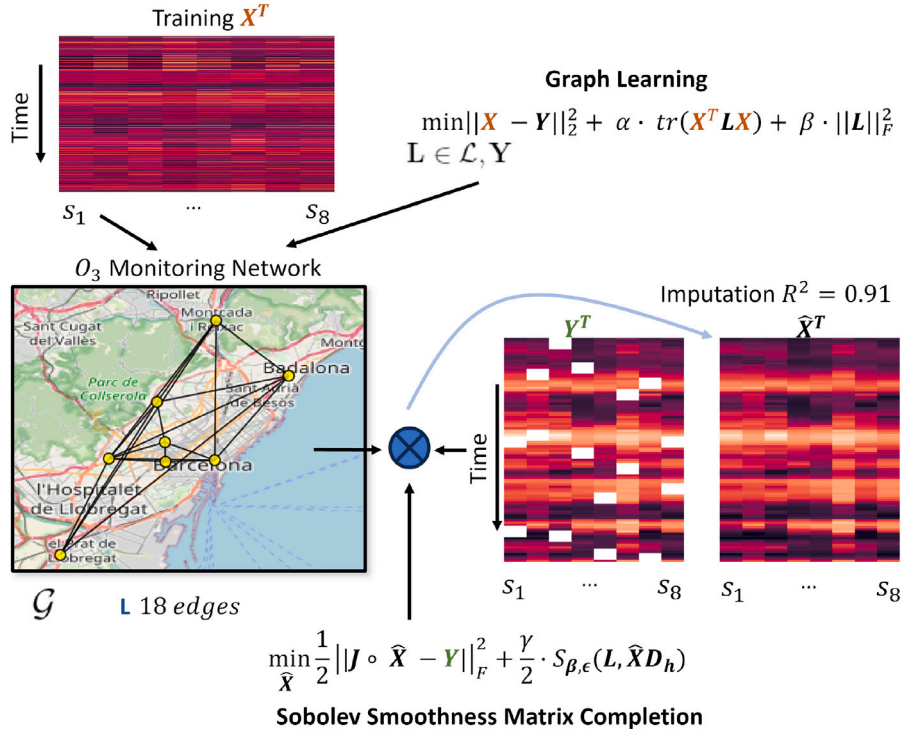

Fig. 14. Example of a missing value imputation scenario in a O_3 monitoring network in the Barcelona area, Spain. A matrix completion model based on the Sobolev smoothness and a learned graph is applied to fill the data gaps, resulting in an imputation R^2 of 0.91.

(see Section 5.1). As mentioned in the previous section, the advantage of GNNs is that they can include a large variety of components to deal with highly complex functions, e.g., spatiotemporal components or attention mechanisms.

4.3.3. Virtual sensing

Virtual sensing or the creation of virtual sensors (also known as soft sensors) is a widespread application in monitoring sensor networks, where estimates for a given phenomenon and location are produced without having a physical sensor, this is the major difference with the missing value imputation task. Common application cases include the creation of virtual sensors to impute missing values, replace readings from faulty sensors, or detect faulty sensors. Therefore, we are estimating the measurements in a given node x_{vs} using the measurements of the other graph nodes $\mathbf{x}_{\mathcal{M}}$. This can be tackled as a graph signal reconstruction or recovery task at a node where there is no underlying physical sensor. More generally, the creation of virtual sensors can be viewed as the signal reconstruction of a node in a graph, the estimation of an unobserved parameter, or the forecasting of multivariate time series. Another example of the application of virtual sensors is the case of dynamic sensor networks where new nodes representing physical sensors and virtual sensors can join or leave the network at any time. In the ML field, this case is also known as out-of-sample imputation while missing value imputation corresponds to in-sample imputation (Jin et al., 2023).

Virtual sensors have been created using graph signal reconstruction and GNNs methods (Chen et al., 2015; Castro-Correa et al., 2023; Jin et al., 2023). For instance, Ferrer-Cid et al. (2022a) use the signal smoothness criterion to estimate virtual sensors for O_3 sensors. In fact, graph signal reconstruction models based on GSP can be used to implement virtual sensors given that the measurements at non-sampled vertices (sensors) are reconstructed using the network measurements (Castro-Correa et al., 2023; Qiu et al., 2017). Lei et al. (2022) propose a Bayesian kernelized matrix factorization model to approach the missing value imputation problem as well as the virtual sensor problem, establishing a connection with Kriging where it is intended to predict in a location where there is no sensor. Graph kernels are used to model spatial relationships.

GNNs have been widely used for virtual sensing applications. Yan et al. (2022) make use of a graph attention network to create virtual sensors for rocket-mounted sensors, which are later used to detect faulty sensors. Cini et al. (2022) also make use of the GRIN to create virtual sensors for an air quality sensor network. Similarly, Wu et al. (2021b) introduce the inductive graph neural network Kriging (IGNNK) capable of generalizing to unobserved nodes, therefore, creating virtual sensors. Moreover, matrix completion using variational graph autoencoders has been carried out to infer air pollutants at a fine-grain scale, thus, creating virtual sensors for unmonitored locations (Do et al., 2019, 2020). Following a similar approach, Calo et al. (2024) introduce a spatial air signal prediction method to estimate the pollution at unobserved locations (street level) using a mean aggregation message passing algorithm in conjunction with a GNN. Indeed, this approach can be seen as obtaining virtual sensors for those locations where the signal is predicted. In terms of phenomena estimation, which can be seen as the implementation of a virtual sensor, Jia et al. (2023) develop a combination of GCN with a temporal convolutional network to estimate the quality of an industrial process. In another interesting example, Niresi et al. (2023) propose a spatiotemporal data fusion GAT to perform air pollution sensor calibration, which can be viewed as obtaining a virtual sensor that fuses measurements from different sensors. In the context of precision agriculture, Vyas and Bandyopadhyay (2022) use a recurrent GNN for soil moisture forecasting from other related environmental parameters. Similarly, Custódio and Prati (2024) evaluate the performance of the spectral temporal GNN, which combines the GDFT and classic layers such as one-dimensional convolutions, to perform soil moisture forecasting from temperature and weather parameters. Zhao et al. (2024) introduce the heterogeneous temporal GNN, which can capture the dynamics of different sensors and include the effect of exogenous variables. Darvishi et al. (2023) use GCN-implemented virtual sensors to detect outliers and replace erroneous sensor readings with those of the virtual sensor if necessary, with a focus on networks feeding digital twins.

Table 5
Examples of virtual sensor applications using graph-based models at the node level.

		Reference	Year	Method	Use case
GSP	Smoothness	Chen et al. (2015)	2015	TV	Air quality sensor network
		Ferrer-Cid et al. (2022a)	2022	Graph learning & signal smoothness	
	Graph filter	Ferrer-Cid et al. (2024b)	2024	Volterra-based graph filter	Environmental monitoring network
Matrix factorization		Lei et al. (2022)	2022	Bayesian graph kernelized matrix factorization	Speed monitoring sensor network
GNN	GCN	Castro-Correa et al. (2023)	2023	Time GNN - GCN and Smoothness regularization	Air quality sensor network
		Custódio and Prati (2024)	2024	Spectral temporal GNN	Precision agriculture
		Darvishi et al. (2023)	2023	Deep recurrent GCN	Water sensor network
		Do et al. (2019)	2019	Variational GAE matrix completion	Air quality sensor network
		Do et al. (2020)	2020		
	Other GNN	Jia et al. (2023)	2023	GCN & temporal convolution	Industrial process network
		Wu et al. (2021b)	2021	IGNNK	Precipitation monitoring network
		Zhao et al. (2024)	2024	Heterogeneous temporal GNN	Industrial sensor network
		Calo et al. (2024)	2024	Message passing algorithm	Air quality sensor network
		Cini et al. (2022)	2022	GRIN	Smart meter sensor network
		Niresi et al. (2023)	2023	GAT & LSTM	Air quality sensor network
		Vyas and Bandyopadhyay (2022)	2022	Recurrent GNN	Precision agriculture
		Yan et al. (2022)	2022	GAN & GAT	Industrial IoT network
		Zhang et al. (2023b)	2023	GLPN	Solar power sensor network

This field is continuously growing, as new graph signal reconstruction techniques, matrix completion techniques, and generalization frameworks are appearing and can be used to create virtual sensors in the context of IoT monitoring sensor networks. Table 5 lists different state-of-the-art models that have been used for the estimation of virtual sensors. As it can be noted, most virtual sensing models are based on GNNs, given their ability to accommodate highly complex spatiotemporal relationships (Dong et al., 2023). Virtual sensors are widely used, whether for outlier detection, sensor replacement, or even parameter estimation at unmonitored sites. The performance of the models depends very much on the nature of the network, whether it is heterogeneous (sensors measuring different phenomena), the amount of data available, or even the smoothness of the measurements. GNNs are the most widely used technique for virtual sensing, transferability and domain generalization properties can be key to their successful application in scenarios where data may be limited. In short, virtual sensing is a key application that allows for uninterrupted measurements and detection of possible anomalies or deviations, which are very important features in scenarios where the data feed digital twins.

4.3.4. Clustering and data aggregation

Node clustering or data aggregation and compression are tasks that are usually carried out in wireless sensor networks. In fact, clustering can be used to group nodes for communication purposes, data compression and aggregation to save energy, or for applying divide-and-conquer-like algorithms. Therefore, the task of discovering K clusters can be seen as finding the map $\mathbf{y} = f_{\mathcal{G}}(\mathbf{X})$, where $\mathbf{X} \in \mathbb{R}^{N\times T}$ is a set of network measurements and $\mathbf{y} = \{1, \dots, K\}^N$ classifies sensors into clusters. In the context of node clustering, deriving from graph spectral theory, the Laplacian eigenvectors have been used for clustering air pollution reference stations (Jabłoński, 2017). Ji et al. (2021) propose a node-level smoothness criterion combined with a self-supervised GCN to perform graph clustering, although not applied, this work gives an idea of possible applications in monitoring sensor networks. Indeed, the Laplacian fielder vector and Laplacian spectral graph theory have been widely used for the clustering of network nodes (Ortega et al., 2018; Muniraju et al., 2017; Thangaramya et al., 2017; Riahi and Gerstoft, 2017).

In the context of data aggregation and compression graph-based techniques have been developed (Manuel et al., 2022). Sensor sampling schedulers have been designed using graph signal reconstruction techniques, so that the best subset of sensors is found to minimize the signal reconstruction error at the unmonitored locations, reducing the overall network energy consumption. Signal reconstruction techniques based on the GDFT have been used for the sensor sampling problem (Rusu and Thompson, 2017). Recently, a graph sampling theory-based model has been developed for dynamic sensor placement, i.e., sensors are not static but can change positions (Nomura et al., 2024).

5. Challenges, limitations, and emerging trends

In this section, we discuss some relevant challenges and limitations that have arisen, as well as identify emerging trends in signal processing over graphs. However, we remark that the applications reviewed, such as virtual sensing, outlier detection, or missing value imputation, are very active research fields.

5.1. Challenges and limitations

In this section, we describe some of the limitations and challenges that graph-based models face in modeling and improving IoT network data quality.

Large-scale IoT monitoring networks: one of the main challenges is the modeling of large-scale sensor networks. In this scenario, a graph can have thousands of nodes, so not only can the training of data quality enhancement models be complex, but the construction of the graph itself can be challenging. In the realm of GSP, linear algebra features, such as vectorization or parallelization of operations, have been used to define operations such as GDFT in high-dimensional scenarios (Sandryhaila and Moura, 2014a). In this line, specialized optimization techniques, such as the alternating direction method of multipliers (ADMM) have been used to solve large-scale modeling GSP-based problems (Han et al., 2019; Dong et al., 2016). Moreover, GNNs provide operations, such as graph sampling or pooling, that allow efficient learning in high-dimensional graphs. In this regard, two fields have emerged that allow for optimizing training in large-scale networks, such as distributed training and the generalization and transference of models to larger networks (Section 5.2.1).

Data availability: is one of the main requirements for training complex data-driven models. This aspect is also relevant in scenarios where IoT platforms are modeled with little data, but similar IoT data is available in other environments. Therefore, several research lines have been developed that study the transference of pre-trained models and the generalization of trained models on small networks to larger networks (Zhou et al., 2020) (Section 5.2.1).

Heterogeneity and dynamic graphs: Another aspect to consider is the complexity and heterogeneity of IoT networks. Although we have focused on static IoT monitoring networks in this survey, more

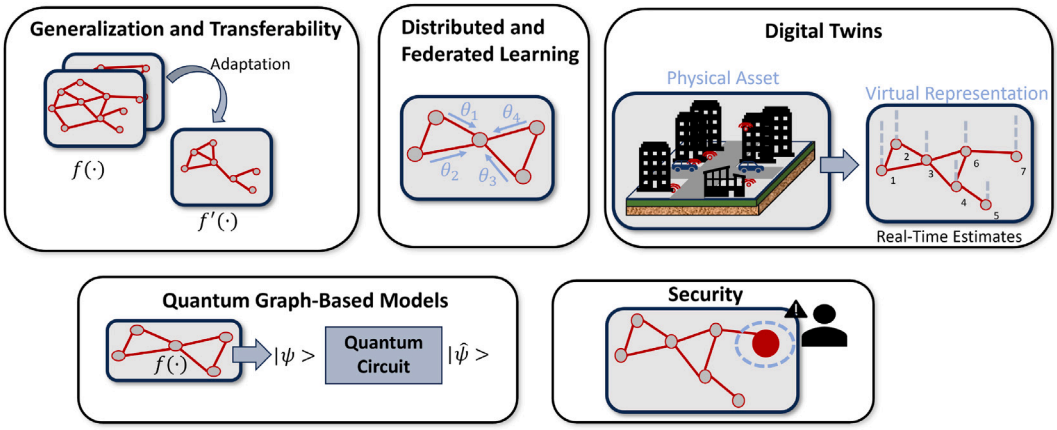

Fig. 15. A number of emerging trends that take advantage of important features offered by graph-based models.

Table 6
Summarization of the different emerging trends highlighted in the paper.

Emerging trends	Graph features
(1) Generalization and transferability	Graph-based models provide intradomain transferability and adaptation properties.
(2) Distributed and federated Learning	Most of graph filters allow a distributed implementation.
(3) Digital twins	Graph-based models can be used to produce estimates for an asset represented by a graph.
(4) Quantum graph-based models	GNNs can be translated into quantum-inspired algorithms.
(5) Security	Graph-based models need to provide robustness against cyberattacks.

advanced networks, such as mobile networks, not only have a variable number of nodes (since a node can enter or leave the network at any time), but also the relationships between nodes can change. Therefore, mobile monitoring networks would require the use of time-varying graphs (Yan et al., 2024). Another aspect that has attracted attention is the assumption of smoothness and homophily used in graph-based models, where connected nodes tend to be similar. This assumption can be compromised in networks where sensors that measure different phenomena coexist or where there are different types of nodes. Recent studies analyze the performance of graph-based models such as GNNs in applications under heterophily (Platonov et al., 2023).

Security: the security of graph models has been one of the most overlooked topics. As previously observed, the robustness of GNNs can be compromised by adversarial attacks (Zhou et al., 2020). Therefore, different attacks have been analyzed and the robustness of GNNs and GSP models has been the subject of study during recent years (Liu et al., 2024a; Sun et al., 2022; Hosseini and Naeini, 2024). This topic is covered in Section 5.2.5. Another related aspect is data privacy and ownership, with federated learning approaches being explored to ensure privacy in IoT applications (Section 5.2.2).

5.2. Emerging trends

In this section, we review some emerging trends in graph-based models for IoT monitoring networks. We show some of the trends that arise in response to some of the challenges and limitations presented in the previous section. Fig. 15 represents the different emerging trends reviewed. Table 6 describes the graph features leveraged in the emerging trends explained in this Section 5.2.

5.2.1. Generalization, transferability, and transfer learning

One of the relevant aspects of graph-based models and more specifically of GNNs is their generalization and transferability, which is a very active research field. When a GNN is trained, the mechanism of message passing or propagation of measurements between nodes is learned, instead of specific relationships between nodes, therefore, models trained on similar networks are prone to be transferable. The generalization and transferability capabilities of GNNs have been highlighted, where ideally models trained on small networks could be transferred to other networks, possibly of larger scale (Ruiz et al., 2023, 2020; Levie et al., 2021; Maskey et al., 2023). The main assumption required is that different networks measure the same phenomenon so that a GNN trained on one network has a similar effect on another network measuring the same phenomenon. The transferability of GNNs is partly inherited from the transferability of graph filters, more specifically spectral graph filters (Levie et al., 2019). For instance, Ruiz et al. (2023) analyze the transferability of graph filters and GNNs, placing special emphasis on the transfer errors depending on the size of the network, where the transfer error is seen to decrease when increasing the original graph size. These ideas are along the lines of transfer learning, where a model, e.g., graph filter or GNN, is pre-trained and then optimized over a new network (Zhu et al., 2021; Gritsenko et al., 2023; Liao et al., 2022). Zhu et al. (2021) develop the ego-graph information maximization to transfer GNNs between graphs. Similarly, graph-based meta-learning techniques can be developed to tackle few-shot learning problems, where a GNN can be trained on a few samples from different data sets and can be applied to a few-shot data set (Zhou et al., 2019; Mandal et al., 2022; Spinelli et al., 2022).

We believe that these approaches will allow the development of more efficient graph-based techniques to carry out different applications in IoT monitoring sensor networks, e.g., missing value imputation or virtual sensing. For instance, in air quality, governmental monitoring networks collect measurements without interruption, so these measurements can be used to create models that can then be applied to low-cost sensor networks, which have no (or little) historical data to train such models.

5.2.2. Distributed and federated learning

One of the relevant aspects of the GSP and graph filters is the fact that by explicitly using the graph shift matrix in their operations, which represents the relationship between nodes, they can be implemented in a distributed manner (Li et al., 2021a). For instance, Xiao et al. (2021) propose the graph Volterra series filter counterpart, which explicitly computes its output based on shifted versions of a graph signal and allows for a distributed approach. Other works have focused on improving the distributed application of graph filters by reducing the

communication and computational resources required (Coutino et al., 2019). The training and inference of distributed models in an IoT environment imply the adoption of an edge computing paradigm that needs to be aware of the resources available in the IoT nodes (Murshed et al., 2021). As in the case of graph filters, GNNs have been the focus of research in their distributed application, given the massive size of networks and data that are used to train such neural networks. Thus, it has been actively investigated how to distribute the training and applications of GNNs in clusters of machines trying to optimize the imbalance in communications and workload, in a parallel fashion (Shao et al., 2024; Besta and Hoefler, 2024; Zheng et al., 2020). For instance, Protogerou et al. (2021) propose a distributed application of a GNN using multi-agents that work cooperatively to perform anomaly detection tasks.

Just as distributed learning has been applied in graph-based applications, the development of federated learning schemes has also been recently applied in graph-based models (Liu et al., 2024b; Xie et al., 2021; Mei et al., 2019; Rasti-Meymandi et al., 2022). In this case, different models are trained independently at each node using the data set available at each node, then these independent models are combined to obtain the final model. This approach is used for applications that require preserving data privacy and ownership, as well as security.

5.2.3. Digital twins

Digital twins are gaining popularity in the academic and industrial environment, and this technology is increasingly intended to be transferred to other fields, e.g., smart cities, e-health, precision agriculture, etc (Thelen et al., 2022). A key component of digital twins is sensing, which complements the physical representation of an entity and provides feedback to improve an underlying physical model. Therefore, sensor networks play a key role in this paradigm, especially the quality of the data they provide.

Graph-based models, and more precisely GNNs, have been used to represent physical entities, therefore, producing digital twins. For instance, in the field of computer communications, GNNs have been used to predict network performance and behavior, being able to detect abnormal conditions (Yu et al., 2023; Wang et al., 2020). Ferriol-Galmés et al. (2022) propose a GNN-based network digital twin to predict network performance and service level agreement metrics.

Applications have also been found in monitoring systems, where digital twins can help to detect different events and trigger the necessary actions. For instance, Roudbari et al. (2024) use a GCRN in conjunction with a data-driven graph to predict water levels in a water monitoring network and feed a digital twin to assess potential flooding. Similarly, Sui et al. (2022) use a graph-based digital twin to model a power system, in the so-called Internet of Energy (IoE), and apply a GCN to detect possible faults and estimate the stability of the system. In the realm of smart healthcare, GNNs have been used to estimate and forecast patients' conditions (Barbiero et al., 2021).

In general, IoT data can be fed into graph-based models that accurately represent physical entities and relationships between monitoring sites to generate useful insights that can later be used to support decision-making processes, perform early warning, predictive maintenance, or anomaly detection. As noted in related publications, graph-based models can represent and provide real-time estimates for an entity or asset. These estimates can be used to detect possible events and failures, or even predict future conditions. Furthermore, as seen in previous sections, graph-powered models are not only effective in estimating digital twins but can also provide solutions to improve the operation of sensor networks that feed digital twins, e.g., through malfunctioning sensor identification and replacement (Darvishi et al., 2023). Therefore, much recent work has focused on improving the data quality and operability of the sensor networks that feed these digital twins.

5.2.4. Quantum graph-based models

Recently with the rise of quantum computing, many quantum algorithms, or so-called quantum-inspired algorithms, have appeared that bring quantum mechanics fundamentals to ML algorithms to provide new properties, e.g., compact parameter representation, and faster training. Thus, much research has focused on developing the quantum analog of known techniques. This phenomenon has also reached graph-based models, particularly in the development of quantum GNNs. For instance, Verdon et al. (2019) introduce the quantum GNNs to represent quantum processes that possess a graph structure. Quantum recurrent GNNs and quantum convolutional GNNs have also been proposed, however, these specific architectures are tailored to a given application, such as graph classification or others (Zheng et al., 2021; Choi et al., 2021). Lately, quantum graph transformers have been proposed, which aim to translate and compute graph transformers in quantum computing primitives and processors (Kollias et al., 2023). In short, we observe how graph-based models such as GNN are translated into quantum-inspired models, i.e., models that bring ideas from quantum computing to improve their classical analog, and other models that aim to use a pure quantum approach with quantum processors. However, we see how a small number of applications have been tackled with quantum graph-based models, e.g., graph classification, and we believe that in the future more applications and tasks will benefit from quantum-inspired and pure-quantum approaches.

5.2.5. Security

Model security and robustness have been some of the most overlooked aspects in the development of graph-based models (Zhou et al., 2020). This is particularly interesting in an era where IoT data is being used in decision-making processes or, as seen above, in digital twin applications. Among the most known attacks are data poisoning, noise injection, or adversarial attacks on GNNs. In the case of GNNs, adversarial attacks affecting the robustness of the methods have been described. In particular, these attacks may include not only the modification of values but also the modification of the graph structure (Guan et al., 2024; Sun et al., 2022; Liu et al., 2024a). In terms of defense, just as attackers can use adversarial samples, adversarial training can be introduced into the GNN training methodology to provide robustness against possible attacks (Guan et al., 2024; Sun et al., 2022). Another defense method is to detect possible modifications of samples, nodes, or edges of the graph, and eliminate possible malicious elements. Similarly, in the context of GSP, GDFT-based techniques have been used to introduce structural adversarial samples, and various GSP methods, such as GDFT or smoothness, have been shown to be useful in detecting maliciously injected data (Liu et al., 2023a; Hosseini and Naeini, 2024).

All in all, the models used must be robust to ensure the integrity of the IoT monitoring network data. In addition, detection models can be introduced to detect possible malicious data injection or modification.

6. Conclusions

This article has reviewed the use of graph-based models, based on frameworks such as GSP, ML over graphs, or GNNs, to improve data quality in IoT monitoring networks. The basics of the different graph-based frameworks that allow building the necessary models in the context of IoT have been presented. Furthermore, a taxonomy has been proposed to identify different applications that can be performed to improve data quality, such as missing value imputation, anomaly detection, or virtual sensing. All these applications can improve different dimensions of IoT data quality such as completeness, accuracy, or consistency. In addition, this taxonomy makes it possible to identify applications that practitioners can use, depending on their scenario, to meet some data quality requirements. The different applications have been classified according to the scope of the application, i.e., whether the objective is at the network level, at the edge or sensor relationship level, and at the node or sensor level. Once the different applications

have been described, different limitations and challenges that may arise when using graph models have been identified. Challenges such as the use of models in large-scale IoT deployments, data availability and heterogeneity, and security. Finally, several emerging trends have been described, such as the development of digital twins using graph-based models or quantum graph models, as well as other areas that seek to provide solutions to the aforementioned limitations, e.g., the generalization and transfer of models, the distributed or federated training, or the security of graph-based models.

CRediT authorship contribution statement

Pau Ferrer-Cid: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Jose M. Barcelo-Ordinas:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation, Funding acquisition, Conceptualization. **Jorge Garcia-Vidal:** Writing – review & editing, Project administration, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by Grant PID2022-138155OB-I00 funded by MCIN/AEI/10.13039/501100011033, Spain and by "ERDF A way of making Europe", CDTI MIG-20221061, regional project, Spain 2021SGR-01059, and by AGAUR 2023 CLIMA 0097.

Data availability

Data will be made available on request.

References

Abu-El-Haija, S., Perozzi, B., Al-Rfou, R., Alemi, A.A., 2018. Watch your step: Learning node embeddings via graph attention. Adv. Neural Inf. Process. Syst. 31.

Ahmad, W.U., Peng, N., Chang, K.-W., 2021. GATE: graph attention transformer encoder for cross-lingual relation and event extraction. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 35, pp. 12462–12470.

Ahn, S.J., Kim, M., 2021. Variational graph normalized autoencoders. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management. pp. 2827–2831.

Allka, X., Ferrer-Cid, P., Barcelo-Ordinas, J.M., Garcia-Vidal, J., 2024. Leveraging spatiotemporal correlations with recurrent autoencoders for sensor anomaly detection. IEEE Internet Things J. http://dx.doi.org/10.1109/JIOT.2024.3416525, 1–1.

Atwood, J., Towsley, D., 2016. Diffusion-convolutional neural networks. Adv. Neural Inf. Process. Syst. 29.

Barbiero, P., Vinas Torne, R., Lió, P., 2021. Graph representation forecasting of patient's medical conditions: Toward a digital twin. Front. Genet. 12, 652907.

Belkin, M., Niyogi, P., 2004. Semi-supervised learning on Riemannian manifolds. Mach. Learn. 56, 209–239.

Besta, M., Hoefler, T., 2024. Parallel and distributed graph neural networks: An in-depth concurrency analysis. IEEE Trans. Pattern Anal. Mach. Intell..

Betancourt, C., Li, C.W., Kleinert, F., Schultz, M.G., 2023. Graph machine learning for improved imputation of missing tropospheric ozone data. Environ. Sci. Technol..

Bianchi, F.M., Grattarola, D., Alippi, C., 2020. Spectral clustering with graph neural networks for graph pooling. In: International Conference on Machine Learning. PMLR, pp. 874–883.

Borgwardt, K., Ghisu, E., Llinares-López, F., O'Bray, L., Rieck, B., et al., 2020. Graph kernels: State-of-the-art and future challenges. Found. Trends® Mach. Learn. 13 (5–6), 531–712.

Bronstein, M.M., Bruna, J., LeCun, Y., Szlam, A., Vandergheynst, P., 2017. Geometric deep learning: going beyond Euclidean data. IEEE Signal Process. Mag. 34 (4), 18–42.

Bui, K.-H.N., Cho, J., Yi, H., 2022. Spatial-temporal graph neural network for traffic forecasting: An overview and open research issues. Appl. Intell. 52 (3), 2763–2774.

Bunch, J.R., Rose, D.J., 2014. Sparse Matrix Computations. Academic Press.

Burago, D., Ivanov, S., Kurylev, Y., 2015. A graph discretization of the Laplace–Beltrami operator. J. Spectr. Theory 4 (4), 675–714.

Calo, S., Bistaffa, F., Jonsson, A., Gómez, V., Viana, M., 2024. Spatial air quality prediction in urban areas via message passing. Eng. Appl. Artif. Intell. 133, 108191. http://dx.doi.org/10.1016/j.engappai.2024.108191, URL https://www.sciencedirect.com/science/article/pii/S095219762400349X.

Castro-Correa, J.A., Giraldo, J.H., Badiey, M., Malliaros, F.D., 2024. Gegenbauer graph neural networks for time-varying signal reconstruction. IEEE Trans. Neural Netw. Learn. Syst..

Castro-Correa, J.A., Giraldo, J.H., Mondal, A., Badiey, M., Bouwmans, T., Malliaros, F.D., 2023. Time-varying signals recovery via graph neural networks. In: ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 1–5.

Chami, I., Abu-El-Haija, S., Perozzi, B., Ré, C., Murphy, K., 2022. Machine learning on graphs: A model and comprehensive taxonomy. J. Mach. Learn. Res. 23 (1), 3840–3903.

Chen, Z., Chen, D., Zhang, X., Yuan, Z., Cheng, X., 2021a. Learning graph structures with transformer for multivariate time-series anomaly detection in IoT. IEEE Internet Things J. 9 (12), 9179–9189.

Chen, S., Eldar, Y.C., Zhao, L., 2021b. Graph unrolling networks: Interpretable neural networks for graph signal denoising. IEEE Trans. Signal Process. 69, 3699–3713.

Chen, S., Sandryhaila, A., Moura, J.M., Kovačević, J., 2015. Signal recovery on graphs: Variation minimization. IEEE Trans. Signal Process. 63 (17), 4609–4624.

Chen, F., Wang, D., Lei, S., He, J., Fu, Y., Lu, C.-T., 2022. Adaptive graph convolutional imputation network for environmental sensor data recovery. Front. Environ. Sci. 10, 2120.

Chiang, W.-L., Liu, X., Si, S., Li, Y., Bengio, S., Hsieh, C.-J., 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 257–266.

Choi, J., Oh, S., Kim, J., 2021. A tutorial on quantum graph recurrent neural network (QGRNN). In: 2021 International Conference on Information Networking. ICOIN, IEEE, pp. 46–49.

Cini, A., Marisca, I., Alippi, C., 2022. Filling the g_ap_s: Multivariate time series imputation by graph neural networks. In: International Conference on Learning Representations. URL https://openreview.net/forum?id=kOu3-S3wJ7.

Cini, A., Zambon, D., Alippi, C., 2023. Sparse graph learning from spatiotemporal time series. J. Mach. Learn. Res. 24 (242), 1–36.

Coutino, M., Isufi, E., Leus, G., 2019. Advances in distributed graph filtering. IEEE Trans. Signal Process. 67 (9), 2320–2333.

Custódio, G., Prati, R.C., 2024. Comparing modern and traditional modeling methods for predicting soil moisture in IoT-based irrigation systems. Smart Agric. Technol. 7, 100397.

Danel, T., Spurek, P., Tabor, J., Śmieja, M., Struski, Ł., Słowik, A., Maziarka, Ł., 2020. Spatial graph convolutional networks. In: International Conference on Neural Information Processing. Springer, pp. 668–675.

Darvishi, H., Ciuonzo, D., Rossi, P.S., 2023. Deep recurrent graph convolutional architecture for sensor fault detection, isolation and accommodation in digital twins. IEEE Sens. J..

Defferrard, M., Bresson, X., Vandergheynst, P., 2016. Convolutional neural networks on graphs with fast localized spectral filtering. Adv. Neural Inf. Process. Syst. 29.

Dehmamy, N., Barabási, A.-L., Yu, R., 2019. Understanding the representation power of graph neural networks in learning graph topology. Adv. Neural Inf. Process. Syst. 32.

Delling, D., Sanders, P., Schultes, D., Wagner, D., 2009. Engineering route planning algorithms. In: Algorithmics of Large and Complex Networks: Design, Analysis, and Simulation. Springer, pp. 117–139.

Deng, A., Hooi, B., 2021. Graph neural network-based anomaly detection in multivariate time series. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 35, pp. 4027–4035.

Deng, L., Lian, D., Huang, Z., Chen, E., 2022. Graph convolutional adversarial networks for spatiotemporal anomaly detection. IEEE Trans. Neural Netw. Learn. Syst. 33 (6), 2416–2428.

Deng, L., Liu, X.-Y., Zheng, H., Feng, X., Chen, Y., 2021. Graph spectral regularized tensor completion for traffic data imputation. IEEE Trans. Intell. Transp. Syst. 23 (8), 10996–11010.

Do, T.H., Nguyen, D.M., Tsiligianni, E., Aguirre, A.L., La Manna, V.P., Pasveer, F., Philips, W., Deligiannis, N., 2019. Matrix completion with variational graph autoencoders: Application in hyperlocal air quality inference. In: ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 7535–7539.

Do, T.H., Tsiligianni, E., Qin, X., Hofman, J., La Manna, V.P., Philips, W., Deligiannis, N., 2020. Graph-deep-learning-based inference of fine-grained air quality from mobile IoT sensors. IEEE Internet Things J. 7 (9), 8943–8955.

Dong, G., Tang, M., Wang, Z., Gao, J., Guo, S., Cai, L., Gutierrez, R., Campbel, B., Barnes, L.E., Boukhechba, M., 2023. Graph neural networks in IoT: a survey. ACM Trans. Sensor Netw. 19 (2), 1–50.

Dong, X., Thanou, D., Frossard, P., Vandergheynst, P., 2016. Learning Laplacian matrix in smooth graph signal representations. IEEE Trans. Signal Process. 64 (23), 6160–6173.

Dong, X., Thanou, D., Rabbat, M., Frossard, P., 2019. Learning graphs from data: A signal representation perspective. IEEE Signal Process. Mag. 36 (3), 44–63.
Dong, X., Thanou, D., Toni, L., Bronstein, M., Frossard, P., 2020. Graph signal processing for machine learning: A review and new perspectives. IEEE Signal Process. Mag. 37 (6), 117–127.
Du, M., Liu, N., Hu, X., 2019. Techniques for interpretable machine learning. Commun. ACM 63 (1), 68–77.
Egilmez, H.E., Ortega, A., 2014. Spectral anomaly detection using graph-based filtering for wireless sensor networks. In: 2014 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 1085–1089.
Egilmez, H.E., Pavez, E., Ortega, A., 2017. Graph learning from data under Laplacian and structural constraints. IEEE J. Sel. Top. Sign. Proces. 11 (6), 825–841.
Feng, Y., Chen, J., Liu, Z., Lv, H., Wang, J., 2022a. Full graph autoencoder for one-class group anomaly detection of IIoT system. IEEE Internet Things J. 9 (21), 21886–21898.
Feng, A., You, C., Wang, S., Tassiulas, L., 2022b. Kergnns: Interpretable graph neural networks with graph kernels. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 36, pp. 6614–6622.
Ferrer-Cid, P., Barcelo-Ordinas, J.M., Garcia-Vidal, J., 2021. Graph learning techniques using structured data for IoT air pollution monitoring platforms. IEEE Internet Things J. 8 (17), 13652–13663.
Ferrer-Cid, P., Barcelo-Ordinas, J.M., Garcia-Vidal, J., 2022a. Data reconstruction applications for IoT air pollution sensor networks using graph signal processing. J. Netw. Comput. Appl. 205, 103434.
Ferrer-Cid, P., Barcelo-Ordinas, J.M., Garcia-Vidal, J., 2022b. Volterra graph-based outlier detection for air pollution sensor networks. IEEE Trans. Netw. Sci. Eng. 9 (4), 2759–2771.
Ferrer-Cid, P., Barcelo-Ordinas, J.M., Garcia-Vidal, J., Ripoll, A., Viana, M., 2020. Multisensor data fusion calibration in IoT air pollution platforms. IEEE Internet Things J. 7 (4), 3124–3132.
Ferrer-Cid, P., Paredes-Ahumada, J.A., Allka, X., Guerrero-Zapata, M., Barcelo-Ordinas, J.M., Garcia-Vidal, J., 2024a. A data-driven framework for air quality sensor networks. IEEE Internet Things Mag. 7 (1), 128–134.
Ferrer-Cid, P., Paredes-Ahumada, J., Barcelo-Ordinas, J.M., Garcia-Vidal, J., 2024b. Virtual sensor-based proxy for black carbon estimation in IoT platforms. Internet Things 27, 101284.
Ferriol-Galmés, M., Suárez-Varela, J., Paillissé, J., Shi, X., Xiao, S., Cheng, X., Barlet-Ros, P., Cabellos-Aparicio, A., 2022. Building a digital twin for network optimization using graph neural networks. Comput. Netw. 217, 109329.
Fortunato, S., 2010. Community detection in graphs. Phys. Rep. 486 (3–5), 75–174.
Francisquini, R., Lorena, A.C., Nascimento, M.C., 2022. Community-based anomaly detection using spectral graph filtering. Appl. Soft Comput. 118, 108489.
Gasteiger, J., Weißenberger, S., Günnemann, S., 2019. Diffusion improves graph learning. Adv. Neural Inf. Process. Syst. 32.
Gavili, A., Zhang, X.-P., 2017. On the shift operator, graph frequency, and optimal filtering in graph signal processing. IEEE Trans. Signal Process. 65 (23), 6303–6318.
Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E., 2017. Neural message passing for quantum chemistry. In: International Conference on Machine Learning. PMLR, pp. 1263–1272.
Giraldo, J.H., Mahmood, A., Garcia-Garcia, B., Thanou, D., Bouwmans, T., 2022. Reconstruction of time-varying graph signals via Sobolev smoothness. IEEE Trans. Signal Inf. Process. Netw. 8, 201–214.
Gopalakrishnan, K., Li, M.Z., Balakrishnan, H., 2019. Identification of outliers in graph signals. In: 2019 IEEE 58th Conference on Decision and Control. CDC, IEEE, pp. 4769–4776.
Gritsenko, A., Shayestehfard, K., Guo, Y., Moharrer, A., Dy, J., Ioannidis, S., 2023. Graph transfer learning. Knowl. Inf. Syst. 65 (4), 1627–1656.
Guan, F., Zhu, T., Zhou, W., Choo, K.-K.R., 2024. Graph neural networks: a survey on the links between privacy and security. Artif. Intell. Rev. 57 (2), 40.
Guerra-García, C., Nikiforova, A., Jiménez, S., Perez-Gonzalez, H.G., Ramírez-Torres, M., Ontañon-García, L., 2023. ISO/IEC 25012-based methodology for managing data quality requirements in the development of information systems: Towards data quality by design. Data Knowl. Eng. 145, 102152.
Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Adv. Neural Inf. Process. Syst. 30.
Han, T., Wada, K., Oguchi, T., 2019. Large-scale traffic data imputation using matrix completion on graphs. In: 2019 IEEE Intelligent Transportation Systems Conference. ITSC, IEEE, pp. 2252–2258.
Han, S., Woo, S.S., 2022. Learning sparse latent graph representations for anomaly detection in multivariate time series. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 2977–2986.
Hang, R., Liu, Q., Song, H., Sun, Y., Zhu, F., Pei, H., 2016. Graph regularized nonlinear ridge regression for remote sensing data analysis. IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens. 10 (1), 277–285.
He, M., Wei, Z., Xu, H., et al., 2021. Bernnet: Learning arbitrary graph spectral filters via bernstein approximation. Adv. Neural Inf. Process. Syst. 34, 14239–14251.
Henaff, M., Bruna, J., LeCun, Y., 2015. Deep convolutional networks on graph-structured data. arXiv preprint arXiv:1506.05163.
Hosseini, R., Naeini, M., 2024. A comprehensive analysis of GSP-based attack detection techniques sensitivity to topology noise in power systems. In: 2024 56th North American Power Symposium. NAPS, IEEE, pp. 1–6.
Hu, W., Pang, J., Liu, X., Tian, D., Lin, C.-W., Vetro, A., 2021. Graph signal processing for geometric data and beyond: Theory and applications. IEEE Trans. Multimed. 24, 3961–3977.
Huang, S., Wang, H., Li, T., Li, T., Xu, Z., 2018. Robust graph regularized nonnegative matrix factorization for clustering. Data Min. Knowl. Discov. 32, 483–503.
Huang, C., Zhang, Q., Huang, J., Yang, L., 2020. Reconstruction of bandlimited graph signals from measurements. Digit. Signal Process. 101, 102728.
Ioannidis, V.N., Romero, D., Giannakis, G.B., 2018. Inference of spatio-temporal functions over graphs via multikernel kriged kalman filtering. IEEE Trans. Signal Process. 66 (12), 3228–3239.
Jabłoński, I., 2017. Graph signal processing in applications to sensor networks, smart grids, and smart cities. IEEE Sens. J. 17 (23), 7659–7666.
Ji, C., Chen, H., Wang, R., Cai, Y., Wu, H., 2021. Smoothness sensor: Adaptive smoothness-transition graph convolutions for attributed graph clustering. IEEE Trans. Cybern. 52 (12), 12771–12784.
Jia, M., Xu, D., Yang, T., Liu, Y., Yao, Y., 2023. Graph convolutional network soft sensor for process quality prediction. J. Process Control 123, 12–25.
Jian, X., Tay, W.P., 2023. Kernel ridge regression for generalized graph signal processing. In: ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 1–5.
Jiang, X., Tian, Z., Li, K., 2021. A graph-based approach for missing sensor data imputation. IEEE Sens. J. 21 (20), 23133–23144.
Jin, M., Koh, H.Y., Wen, Q., Zambon, D., Alippi, C., Webb, G.I., King, I., Pan, S., 2023. A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection. arXiv preprint arXiv:2307.03759.
Kalofolias, V., 2016. How to learn a graph from smooth signals. In: Artificial Intelligence and Statistics. PMLR, pp. 920–929.
Kipf, T.N., Welling, M., 2016a. Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations.
Kipf, T.N., Welling, M., 2016b. Variational graph auto-encoders. In: NIPS Workshop on Bayesian Deep Learning.
Kök, İ., Okay, F.Y., Muyanlı, Ö., Özdemir, S., 2023. Explainable artificial intelligence (xai) for internet of things: a survey. IEEE Internet Things J..
Kollias, G., Kalantzis, V., Salonidis, T., Ubaru, S., 2023. Quantum graph transformers. In: ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 1–5.
Lei, M., Labbe, A., Wu, Y., Sun, L., 2022. Bayesian kernelized matrix factorization for spatiotemporal traffic data imputation and kriging. IEEE Trans. Intell. Transp. Syst. 23 (10), 18962–18974.
Leus, G., Marques, A.G., Moura, J.M., Ortega, A., Shuman, D.I., 2023. Graph signal processing: History, development, impact, and outlook. IEEE Signal Process. Mag. 40 (4), 49–60.
Levie, R., Huang, W., Bucci, L., Bronstein, M., Kutyniok, G., 2021. Transferability of spectral graph convolutional neural networks. J. Mach. Learn. Res. 22 (1), 12462–12520.
Levie, R., Isufi, E., Kutyniok, G., 2019. On the transferability of spectral graph filters. In: 2019 13th International Conference on Sampling Theory and Applications. SampTA, IEEE, pp. 1–5.
Levie, R., Monti, F., Bresson, X., Bronstein, M.M., 2018. Cayleynets: Graph convolutional neural networks with complex rational spectral filters. IEEE Trans. Signal Process. 67 (1), 97–109.
Lewenfus, G., Alves Martins, W., Chatzinotas, S., Ottersten, B., 2019. On the use of vertex-frequency analysis for anomaly detection in graph signals. In: Anais do XXXVII Simpósio Brasileiro de Telecomunicações e Processamento de Sinais. SBrT 2019, pp. 1–5.
Li, Q., Coutino, M., Leus, G., Christensen, M.G., 2021a. Privacy-preserving distributed graph filtering. In: 2020 28th European Signal Processing Conference. EUSIPCO, IEEE, pp. 2155–2159.
Li, R.-Y., Guo, Y., Zhang, B., 2023a. Adaptive kernel graph nonnegative matrix factorization. Information 14 (4), 208.
Li, Z., Kang, Y., Feng, D., Wang, X.-M., Lv, W., Chang, J., Zheng, W.X., 2020a. Semi-supervised learning for lithology identification using Laplacian support vector machine. J. Pet. Sci. Eng. 195, 107510.
Li, S., Lai, S., Jiang, Y., Wang, W., Yi, Y., 2021b. Graph regularized deep sparse representation for unsupervised anomaly detection. Comput. Intell. Neurosci. 2021 (1), 4026132.
Li, Y., Xie, S., Wan, Z., Lv, H., Song, H., Lv, Z., 2023b. Graph-powered learning methods in the internet of things: A survey. Mach. Learn. Appl. 11, 100441.
Li, J., Yu, J., Li, J., Zhang, H., Zhao, K., Rong, Y., Cheng, H., Huang, J., 2020b. Dirichlet graph variational autoencoder. Adv. Neural Inf. Process. Syst. 33, 5274–5283.
Li, Y., Yu, R., Shahabi, C., Liu, Y., 2018. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In: International Conference on Learning Representations.
Li, R., Yuan, X., Radfar, M., Marendy, P., Ni, W., O'Brien, T.J., Casillas-Espinosa, P.M., 2021c. Graph signal processing, graph neural network and graph learning on biological data: a systematic review. IEEE Rev. Biomed. Eng. 16, 109–135.
Li, Y., Zemel, R., Brockschmidt, M., Tarlow, D., 2016. Gated graph sequence neural networks. In: Proceedings of ICLR'16.
Li, M., Zhu, Z., 2021. Spatial-temporal fusion graph neural networks for traffic flow forecasting. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 35, pp. 4189–4196.

Liao, T., Zhao, J., Liu, Y., Ivanov, K., Xiong, J., Yan, Y., 2022. Deep transfer learning with graph neural network for sensor-based human activity recognition. In: 2022 IEEE International Conference on Bioinformatics and Biomedicine. BIBM, IEEE, pp. 2445–2452.

Lin, X., Wang, H., Guo, J., Mei, G., 2022. A deep learning approach using graph neural networks for anomaly detection in air quality data considering spatiotemporal correlations. IEEE Access 10, 94074–94088.

Liu, D., Hu, W., Li, X., 2023a. Point cloud attacks in graph spectral domain: When 3d geometry meets graph signal processing. IEEE Trans. Pattern Anal. Mach. Intell..

Liu, A., Li, W., Li, T., Li, B., Huang, H., Zhou, P., 2024a. Towards inductive robustness: Distilling and fostering wave-induced resonance in transductive GCNs against graph adversarial attacks. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 38, pp. 13855–13863.

Liu, C., Nitschke, P., Williams, S.P., Zowghi, D., 2020a. Data quality and the internet of things. Computing 102 (2), 573–599.

Liu, R., Xing, P., Deng, Z., Li, A., Guan, C., Yu, H., 2024b. Federated graph neural networks: Overview, techniques, and challenges. IEEE Trans. Neural Netw. Learn. Syst..

Liu, H., Xu, X., Li, E., Zhang, S., Li, X., 2021. Anomaly detection with representative neighbors. IEEE Trans. Neural Netw. Learn. Syst. 34 (6), 2831–2841.

Liu, Y., Yang, L., You, K., Guo, W., Wang, W., 2019. Graph learning based on spatiotemporal smoothness for time-varying graph signal. IEEE Access 7, 62372–62386.

Liu, S., Yao, S., Huang, Y., Liu, D., Shao, H., Zhao, Y., Li, J., Wang, T., Wang, R., Yang, C., et al., 2020b. Handling missing sensors in topology-aware iot applications with gated graph neural network. Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. 4 (3), 1–31.

Liu, C., Zhan, Y., Wu, J., Li, C., Du, B., Hu, W., Liu, T., Tao, D., 2023b. Graph pooling for graph neural networks: progress, challenges, and opportunities. In: Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence. pp. 6712–6722.

Liu, Z., Zhou, J., 2022. Introduction to Graph Neural Networks. Springer Nature.

López, E., Vionnet, C., Ferrer-Cid, P., Barcelo-Ordinas, J.M., Garcia-Vidal, J., Contini, G., Prodolliet, J., Maiztegui, J., 2022. A low-power iot device for measuring water table levels and soil moisture to ease increased crop yields. Sensors 22 (18), 6840.

Ma, Y., Wang, S., Aggarwal, C.C., Yin, D., Tang, J., 2019. Multi-dimensional graph convolutional networks. In: Proceedings of the 2019 Siam International Conference on Data Mining. SIAM, pp. 657–665.

Mandal, D., Medya, S., Uzzi, B., Aggarwal, C., 2022. Metalearning with graph neural networks: Methods and applications. ACM SIGKDD Explor. Newsl. 23 (2), 13–22.

Mansouri, T., Sadeghi Moghadam, M.R., Monshizadeh, F., Zareravasan, A., 2023. IoT data quality issues and potential solutions: a literature review. Comput. J. 66 (3), 615–625.

Manuel, E.M., Pankajakshan, V., Mohan, M.T., 2022. Data aggregation in low-power wireless sensor networks with discrete transmission ranges: Sensor signal aggregation over graph. IEEE Sens. J. 22 (21), 21135–21144.

Marisca, I., Cini, A., Alippi, C., 2022. Learning to reconstruct missing data from spatiotemporal graphs with sparse observations. Adv. Neural Inf. Process. Syst. 35, 32069–32082.

Maskey, S., Levie, R., Kutyniok, G., 2023. Transferability of graph neural networks: an extended graphon approach. Appl. Comput. Harmon. Anal. 63, 48–83.

Mateos, G., Segarra, S., Marques, A.G., Ribeiro, A., 2019. Connecting the dots: Identifying network structure via graph signal processing. IEEE Signal Process. Mag. 36 (3), 16–43.

Mei, G., Guo, Z., Liu, S., Pan, L., 2019. Sgnn: A graph neural network based federated learning approach by hiding structure. In: 2019 IEEE International Conference on Big Data. Big Data, IEEE, pp. 2560–2568.

Mei, J., Moura, J.M., 2016. Signal processing on graphs: Causal modeling of unstructured data. IEEE Trans. Signal Process. 65 (8), 2077–2092.

Melacci, S., Belkin, M., 2011. Laplacian support vector machines trained in the primal. J. Mach. Learn. Res. 12 (3).

Miele, E.S., Bonacina, F., Corsini, A., 2022. Deep anomaly detection in horizontal axis wind turbines using graph convolutional autoencoders for multivariate time series. Energy AI 8, 100145.

Molnar, C., 2020. Interpretable Machine Learning. Lulu. com.

Mondal, A., Das, M., Chatterjee, A., Venkateswaran, P., 2022. Recovery of missing sensor data by reconstructing time-varying graph signals. In: 2022 30th European Signal Processing Conference. EUSIPCO, IEEE, pp. 2181–2185.

Mu, J., Song, P., Liu, X., Li, S., 2023. Dual-graph regularized concept factorization for multi-view clustering. Expert Syst. Appl. 223, 119949.

Muniraju, G., Zhang, S., Tepedelenlioglu, C., Banavar, M.K., Spanias, A., Vargas-Rosales, C., Villalpando-Hernandez, R., 2017. Location based distributed spectral clustering for wireless sensor networks. In: 2017 Sensor Signal Processing for Defence Conference. SSPD, IEEE, pp. 1–5.

Murshed, M.S., Murphy, C., Hou, D., Khan, N., Ananthanarayanan, G., Hussain, F., 2021. Machine learning at the network edge: A survey. ACM Comput. Surv. 54 (8), 1–37.

Newman, M.E., Watts, D.J., Strogatz, S.H., 2002. Random graph models of social networks. Proc. Natl. Acad. Sci. 99 (suppl_1), 2566–2572.

Niresi, K.F., Zhao, M., Bissig, H., Baumann, H., Fink, O., 2023. Spatial-temporal graph attention fuser for calibration in IoT air pollution monitoring systems. In: 2023 IEEE SENSORS. IEEE, pp. 01–04.

Nomura, S., Hara, J., Higashi, H., Tanaka, Y., 2024. Dynamic sensor placement based on sampling theory for graph signals. IEEE Open J. Signal Process..

Ortega, A., Frossard, P., Kovačević, J., Moura, J.M., Vandergheynst, P., 2018. Graph signal processing: Overview, challenges, and applications. Proc. IEEE 106 (5), 808–828.

Pal, B., Jenamani, M., 2018. Kernelized probabilistic matrix factorization for collaborative filtering: exploiting projected user and item graph. In: Proceedings of the 12th ACM Conference on Recommender Systems. pp. 437–440.

Pan, S., Hu, R., Long, G., Jiang, J., Yao, L., Zhang, C., 2018. Adversarially regularized graph autoencoder for graph embedding. In: International Joint Conference on Artificial Intelligence 2018. Association for the Advancement of Artificial Intelligence (AAAI), pp. 2609–2615.

Pavlopoulos, G.A., Secrier, M., Moschopoulos, C.N., Soldatos, T.G., Kossida, S., Aerts, J., Schneider, R., Bagos, P.G., 2011. Using graph theory to analyze biological networks. BioData Min. 4, 1–27.

Pilavcı, Y.Y., Amblard, P.-O., Barthelmé, S., Tremblay, N., 2021. Graph tikhonov regularization and interpolation via random spanning forests. IEEE Trans. Signal Inf. Process. Netw. 7, 359–374.

Platonov, O., Kuznedelev, D., Diskin, M., Babenko, A., Prokhorenkova, L., 2023. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? arXiv preprint arXiv:2302.11640.

Protogerou, A., Papadopoulos, S., Drosou, A., Tzovaras, D., Refanidis, I., 2021. A graph neural network method for distributed anomaly detection in IoT. Evol. Syst. 12, 19–36.

Puny, O., Lim, D., Kiani, B., Maron, H., Lipman, Y., 2023. Equivariant polynomials for graph neural networks. In: International Conference on Machine Learning. PMLR, pp. 28191–28222.

Qiu, K., Mao, X., Shen, X., Wang, X., Li, T., Gu, Y., 2017. Time-varying graph signal reconstruction. IEEE J. Sel. Top. Sign. Proces. 11 (6), 870–883.

Rasti-Meymandi, A., Sheikholeslami, S.M., Abouei, J., Plataniotis, K.N., 2022. Graph federated learning for CIoT devices in smart home applications. IEEE Internet Things J. 10 (8), 7062–7079.

Ren, J., Xia, F., Lee, I., Noori Hoshyar, A., Aggarwal, C., 2023. Graph learning for anomaly analytics: Algorithms, applications, and challenges. ACM Trans. Intell. Syst. Technol. 14 (2), 1–29.

Riahi, N., Gerstoft, P., 2017. Using graph clustering to locate sources within a dense sensor array. Signal Process. 132, 110–120.

Ricaud, B., Borgnat, P., Tremblay, N., Gonçalves, P., Vandergheynst, P., 2019. Fourier could be a data scientist: From graph Fourier transform to signal processing on graphs. C. R. Phys. 20 (5), 474–488.

Romero, D., Ma, M., Giannakis, G.B., 2016. Kernel-based reconstruction of graph signals. IEEE Trans. Signal Process. 65 (3), 764–778.

Roudbari, N.S., Punekar, S.R., Patterson, Z., Eicker, U., Poullis, C., 2024. From data to action in flood forecasting leveraging graph neural networks and digital twin visualization. Sci. Rep. 14 (1), 18571.

Ruiz, L., Chamon, L., Ribeiro, A., 2020. Graphon neural networks and the transferability of graph neural networks. Adv. Neural Inf. Process. Syst. 33, 1702–1712.

Ruiz, L., Chamon, L.F., Ribeiro, A., 2023. Transferability properties of graph neural networks. IEEE Trans. Signal Process..

Rusu, C., Thompson, J., 2017. Node sampling by partitioning on graphs via convex optimization. In: 2017 Sensor Signal Processing for Defence Conference. SSPD, IEEE, pp. 1–5.

Sahbi, H., 2021. Kernel-based graph convolutional networks. In: 2020 25th International Conference on Pattern Recognition. ICPR, IEEE, pp. 4887–4894.

Salha, G., Hennequin, R., Vazirgiannis, M., 2019. Keep it simple: Graph autoencoders without graph convolutional networks. In: Workshop on Graph Representation Learning, 33rd Conference on Neural Information Processing Systems. NIPS.

Sandryhaila, A., Moura, J.M., 2013a. Discrete signal processing on graphs. IEEE Trans. Signal Process. 61 (7), 1644–1656.

Sandryhaila, A., Moura, J.M., 2013b. Discrete signal processing on graphs: Graph fourier transform. In: 2013 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE, pp. 6167–6170.

Sandryhaila, A., Moura, J.M., 2014a. Big data analysis with signal processing on graphs: Representation and processing of massive data sets with irregular structure. IEEE Signal Process. Mag. 31 (5), 80–90. http://dx.doi.org/10.1109/MSP.2014.2329213.

Sandryhaila, A., Moura, J.M., 2014b. Discrete signal processing on graphs: Frequency analysis. IEEE Trans. Signal Process. 62 (12), 3042–3054.

Shafin, S.S., Karmakar, G., Mareels, I., Balasubramanian, V., Kolluri, R.R., 2024. Sensor self-declaration of numeric data reliability in internet of things. IEEE Trans. Reliab..

Shao, Y., Li, H., Gu, X., Yin, H., Li, Y., Miao, X., Zhang, W., Cui, B., Chen, L., 2024. Distributed graph neural network training: A survey. ACM Comput. Surv. 56 (8), 1–39.

Shuman, D.I., Narang, S.K., Frossard, P., Ortega, A., Vandergheynst, P., 2013. The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains. IEEE Signal Process. Mag. 30 (3), 83–98.

Simonovsky, M., Komodakis, N., 2018. Graphvae: Towards generation of small graphs using variational autoencoders. In: Artificial Neural Networks and Machine Learning–ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4-7, 2018, Proceedings, Part I 27. Springer, pp. 412–422.

Skarding, J., Gabrys, B., Musial, K., 2021. Foundations and modeling of dynamic networks using dynamic graph neural networks: A survey. IEEE Access 9, 79143–79168.

Smola, A.J., Kondor, R., 2003. Kernels and regularization on graphs. In: Learning Theory and Kernel Machines: 16th Annual Conference on Learning Theory and 7th Kernel Workshop, COLT/Kernel 2003, Washington, DC, USA, August 24-27, 2003. Proceedings. Springer, pp. 144–158.

Song, Z., Yang, X., Xu, Z., King, I., 2022. Graph-based semi-supervised learning: A comprehensive review. IEEE Trans. Neural Netw. Learn. Syst..

Spinelli, I., Scardapane, S., Uncini, A., 2020. Missing data imputation with adversarially-trained graph convolutional networks. Neural Netw. 129, 249–260.

Spinelli, I., Scardapane, S., Uncini, A., 2022. A meta-learning approach for training explainable graph neural networks. IEEE Trans. Neural Netw. Learn. Syst..

Stanković, L., Mandic, D., Daković, M., Brajović, M., Scalzo, B., Li, S., Constantinides, A.G., et al., 2020a. Data analytics on graphs part I: Graphs and spectra on graphs. Found. Trends® Mach. Learn. 13 (1), 1–157.

Stanković, L., Mandic, D., Daković, M., Brajović, M., Scalzo, B., Li, S., Constantinides, A.G., et al., 2020b. Data analytics on graphs part III: Machine learning on graphs, from graph topology to applications. Found. Trends® Mach. Learn. 13 (4), 332–530.

Su, L., Han, Z., Zhao, J., Wang, W., 2024. Graph-frequency domain Kalman filtering for industrial pipe networks subject to measurement outliers. IEEE Trans. Ind. Inform..

Subramanya, A., Talukdar, P.P., 2022. Graph-Based Semi-Supervised Learning. Springer Nature.

Sui, L., Guan, X., Cui, C., Jiang, H., Pan, H., Ohtsuki, T., 2022. Graph learning empowered situation awareness in internet of energy with graph digital twin. IEEE Trans. Ind. Inform. 19 (5), 7268–7277.

Sun, L., Dou, Y., Yang, C., Zhang, K., Wang, J., Philip, S.Y., He, L., Li, B., 2022. Adversarial attack and defense on graph data: A survey. IEEE Trans. Knowl. Data Eng. 35 (8), 7693–7711.

Tang, J., Li, J., Gao, Z., Li, J., 2022. Rethinking graph neural networks for anomaly detection. In: International Conference on Machine Learning. PMLR, pp. 21076–21089.

Teh, H.Y., Kempa-Liehr, A.W., Wang, K.I.-K., 2020. Sensor data quality: A systematic review. J. Big Data 7 (1), 11.

Thangaramya, K., Logambigai, R., SaiRamesh, L., Kulothungan, K., Kannan, A., Ganapathy, S., 2017. An energy efficient clustering approach using spectral graph theory in wireless sensor networks. In: 2017 Second International Conference on Recent Trends and Challenges in Computational Models. ICRTCCM, IEEE, pp. 126–129.

Thelen, A., Zhang, X., Fink, O., Lu, Y., Ghosh, S., Youn, B.D., Todd, M.D., Mahadevan, S., Hu, C., Hu, Z., 2022. A comprehensive review of digital twin—part 1: modeling and twinning enabling technologies. Struct. Multidiscip. Optim. 65 (12), 354.

Tong, A., van Dijk, D., Stanley, III, J.S., Amodio, M., Yim, K., Muhle, R., Noonan, J., Wolf, G., Krishnaswamy, S., 2020. Interpretable neuron structuring with graph spectral regularization. In: Advances in Intelligent Data Analysis XVIII: 18th International Symposium on Intelligent Data Analysis, IDA 2020, Konstanz, Germany, April 27–29, 2020, Proceedings 18. Springer, pp. 509–521.

Van De Ville, D., Demesmaeker, R., Preti, M.G., 2017. When slepian meets fiedler: Putting a focus on the graph spectrum. IEEE Signal Process. Lett. 24 (7), 1001–1004.

Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al., 2017. Graph attention networks. Statistics 1050 (20), 10–48550.

Venkitaraman, A., Chatterjee, S., Händel, P., 2019. Predicting graph signals using kernel regression where the input signal is agnostic to a graph. IEEE Trans. Signal Inf. Process. Netw. 5 (4), 698–710.

Venkitaraman, A., Chatterjee, S., Handel, P., 2020. Gaussian processes over graphs. In: ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing. ICASSP, IEEE, pp. 5640–5644.

Verdon, G., McCourt, T., Luzhnica, E., Singh, V., Leichenauer, S., Hidary, J., 2019. Quantum graph neural networks. arXiv preprint arXiv:1909.12264.

Vishwanathan, S.V.N., Schraudolph, N.N., Kondor, R., Borgwardt, K.M., 2010. Graph kernels. J. Mach. Learn. Res. 11, 1201–1242.

Vyas, A., Bandyopadhyay, S., 2022. Dynamic structure learning through graph neural network for forecasting soil moisture in precision agriculture. In: Raedt, L.D. (Ed.), Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence. IJCAI-22, International Joint Conferences on Artificial Intelligence Organization, pp. 5185–5191. http://dx.doi.org/10.24963/ijcai.2022/720, AI for Good.

Wang, H., Leskovec, J., 2020. Unifying graph convolutional neural networks and label propagation. arXiv preprint arXiv:2002.06755.

Wang, C., Pan, S., Long, G., Zhu, X., Jiang, J., 2017. Mgae: Marginalized graph autoencoder for graph clustering. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. pp. 889–898.

Wang, H., Wang, J., Wang, J., Zhao, M., Zhang, W., Zhang, F., Xie, X., Guo, M., 2018. Graphgan: Graph representation learning with generative adversarial nets. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 32.

Wang, H., Wu, Y., Min, G., Miao, W., 2020. A graph neural network-based digital twin for network slicing management. IEEE Trans. Ind. Inform. 18 (2), 1367–1376.

Wang, Y., Zhang, B., Ma, J., Jin, Q., 2023. Artificial intelligence of things (AIoT) data acquisition based on graph neural networks: A systematical review. Concurr. Comput.: Pract. Exper. 35 (23), e7827.

Wu, L., Cui, P., Pei, J., Zhao, L., Guo, X., 2022. Graph neural networks: foundation, frontiers and applications. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 4840–4841.

Wu, Y., Dai, H.-N., Tang, H., 2021a. Graph neural networks for anomaly detection in industrial internet of things. IEEE Internet Things J. 9 (12), 9214–9231.

Wu, D.Y., Lin, T.-H., Zhang, X.-R., Chen, C.-P., Chen, J.-H., Chen, H.-H., 2023. Detecting inaccurate sensors on a large-scale sensor network using centralized and localized graph neural networks. IEEE Sens. J..

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Philip, S.Y., 2020. A comprehensive survey on graph neural networks. IEEE Trans. Neural Netw. Learn. Syst. 32 (1), 4–24.

Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., Weinberger, K., 2019. Simplifying graph convolutional networks. In: International Conference on Machine Learning. PMLR, pp. 6861–6871.

Wu, Y., Zhuang, D., Labbe, A., Sun, L., 2021b. Inductive graph neural networks for spatiotemporal kriging. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 35, pp. 4478–4485.

Xia, F., Sun, K., Yu, S., Aziz, A., Wan, L., Pan, S., Liu, H., 2021. Graph learning: A survey. IEEE Trans. Artif. Intell. 2 (2), 109–127.

Xiao, Z., Fang, H., Wang, X., 2020a. Anomalous IoT sensor data detection: An efficient approach enabled by nonlinear frequency-domain graph analysis. IEEE Internet Things J. 8 (5), 3812–3821.

Xiao, Z., Fang, H., Wang, X., 2020b. Nonlinear polynomial graph filter for anomalous IoT sensor detection and localization. IEEE Internet Things J. 7 (6), 4839–4848.

Xiao, Z., Fang, H., Wang, X., 2021. Distributed nonlinear polynomial graph filter and its output graph spectrum: Filter analysis and design. IEEE Trans. Signal Process. 69, 1725–1739.

Xie, L., Pi, D., Zhang, X., Chen, J., Luo, Y., Yu, W., 2021. Graph neural network approach for anomaly detection. Measurement 180, 109546.

Xu, B., Shen, H., Cao, Q., Qiu, Y., Cheng, X., 2019. Graph wavelet neural network. In: International Conference on Learning Representations.

Xu, A., Wu, T., Zhang, Y., Hu, Z., Jiang, Y., 2021. Graph-based time series edge anomaly detection in smart grid. In: 2021 7th IEEE Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing,(HPSC) and IEEE Intl Conference on Intelligent Data and Security. IDS, IEEE, pp. 1–6.

Yan, Y., Hou, J., Song, Z., Kuruoglu, E.E., 2024. Signal processing over time-varying graphs: A systematic review. arXiv preprint arXiv:2412.00462.

Yan, H., Wang, J., Chen, J., Liu, Z., Feng, Y., 2022. Virtual sensor-based imputed graph attention network for anomaly detection of equipment with incomplete data. J. Manuf. Syst. 63, 52–63.

Yang, M., Coutino, M., Leus, G., Isufi, E., 2021. Node-adaptive regularization for graph signal reconstruction. IEEE Open J. Signal Process. 2, 85–98.

Yang, J., Yue, Z., 2022. Learning hierarchical spatial-temporal graph representations for robust multivariate industrial anomaly detection. IEEE Trans. Ind. Inform..

Yu, P., Zhang, J., Fang, H., Li, W., Feng, L., Zhou, F., Xiao, P., Guo, S., 2023. Digital twin driven service self-healing with graph neural networks in 6G edge networks. IEEE J. Sel. Areas Commun..

Zhang, S., Chen, J., Chen, J., Chen, X., Huang, H., 2023a. Data imputation in iot using spatio-temporal variational auto-encoder. Neurocomputing 529, 23–32.

Zhang, C., Florêncio, D., Chou, P.A., 2015. Graph signal processing-a probabilistic framework. In: Microsoft Res., Redmond, WA, USA. Tech. Rep. MSR-TR-2015-31.

Zhang, W., Li, G., Tang, J., Li, J., Tsung, F., 2023b. Missing data imputation with graph Laplacian pyramid network. arXiv preprint arXiv:2304.04474.

Zhang, S., Tong, H., Xu, J., Maciejewski, R., 2019. Graph convolutional networks: a comprehensive review. Comput. Soc. Netw. 6 (1), 1–23.

Zhang, Y., Zhang, Q., Zhang, Y., Zhu, Z., 2023c. VGAE-AMF: A novel topology reconstruction algorithm for invulnerability of ocean wireless sensor networks based on graph neural network. J. Mar. Sci. Eng. 11 (4), 843.

Zhao, M., Taal, C., Baggerohr, S., Fink, O., 2024. Graph neural networks for virtual sensing in complex systems: Addressing heterogeneous temporal dynamics. arXiv preprint arXiv:2407.18691.

Zheng, J., Gao, Q., Lü, Y., 2021. Quantum graph convolutional neural networks. In: 2021 40th Chinese Control Conference. CCC, IEEE, pp. 6335–6340.

Zheng, D., Ma, C., Wang, M., Zhou, J., Su, Q., Song, X., Gan, Q., Zhang, Z., Karypis, G., 2020. Distdgl: distributed graph neural network training for billion-scale graphs. In: 2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms. IA3, IEEE, pp. 36–44.

Zhou, X., Belkin, M., 2014. Semi-supervised learning. In: Academic Press Library in Signal Processing. Vol. 1, Elsevier, pp. 1239–1269.

Zhou, F., Cao, C., Zhang, K., Trajcevski, G., Zhong, T., Geng, J., 2019. Meta-gnn: On few-shot node classification in graph meta-learning. In: Proceedings of the 28th ACM International Conference on Information and Knowledge Management. pp. 2357–2360.

Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M., 2020. Graph neural networks: A review of methods and applications. AI Open 1, 57–81.

Zhou, B., Jiang, Y., Wang, Y., Liang, J., Gao, J., Pan, S., Zhang, X., 2023. Robust graph representation learning for local corruption recovery. In: Proceedings of the ACM Web Conference 2023. pp. 438–448.

Zhu, Q., Yang, C., Xu, Y., Wang, H., Zhang, C., Han, J., 2021. Transfer learning of graph neural networks with ego-graph information maximization. Adv. Neural Inf. Process. Syst. 34, 1766–1779.

Pau Ferrer-Cid is a postdoc researcher at the Statistical Analysis of Networks and Systems (SANS) research group, Universitat Politecnica de Catalunya (UPC). He holds a B.Sc in Computer Science, a M.Sc in Data Science, and a Ph.D. from the UPC. His main research interests are the development of novel data analysis methods for IoT platforms and artificial intelligence of things (AIoT).

Jose M. Barcelo-Ordinas is full professor at Universitat Politecnica de Catalunya (UPC). He holds a Ph.D. and B.Sc+M.Sc in Telecommunication Engineering and a B.Sc+M.Sc in Mathematics. His current research areas are assessing the quality of data in IoT monitoring networks, and the design of digital twins.

Jorge Garcia-Vidal holds a Ph.D. in Telecommunication Engineering (1992). He is a full professor at the Computer Architecture Department of UPC since 2003, and a senior researcher at Barcelona Supercomputing Center (BSC-CNS) since 2012. His main current research interest is in problems related to the capture, processing and statistical analysis of sensor data.